```
{
  "dataType": "ChatOpenAI",       // тип компонента
  "id": "abc-123",                // ID ноды
  "output_types": ["Message"],    // типы выхода
  "name": "response"              // имя порта
  // также: baseClasses (legacy), conditionalPath (условный роутинг)
}
```

Target handle (левая сторона, вход компонента):

```
{
  "type": "str",                  // тип поля
  "fieldName": "input_value",     // имя поля в template
  "id": "def-456",                // ID ноды
  "inputTypes": ["Message"]       // какие типы принимает
  // также: output_types (для loop), proxy, name
}
```

Эти handle сериализуются в строку через специальную функцию scapedJSONStringfy, которая заменяет кавычки на символ "oe" чтобы не конфликтовать с атрибутами React Flow.

6.5.1.2 1.2 Что хранится в ноде

Каждая нода хранит полный снимок определения компонента на момент сохранения:

- **data.node.template** - словарь входных полей (type, input_types, value, show, advanced и т.д.)
- **data.node.outputs** - массив выходов (name, types, method, selected, display_name)
- **data.node.template.code.value** - Python-код компонента на момент сохранения
- **data.type** - тип компонента (например "ChatOpenAI")
- **data.selected_output** - какой выходной порт сейчас выбран

6.5.1.3 1.3 Хранение в базе данных

Flow хранится в таблице flow как JSON-колонка:

```
# src/backend/base/langflow/services/database/models/flow/model.py
class Flow(FlowBase, table=True):
    id: UUID
    data: dict | None = Field(sa_column=Column(JSON))  # {"nodes": [...], "edges":
[...]}
    user_id: UUID | None
    name: str
    # + folder_id, tags, locked, endpoint_name и другие поля
```

Поле data содержит полный граф - ноды и связи в виде JSON-снимка. Никакой нормализации - бэкенд хранит ровно то, что прислал фронтенд.

6.5.1.4 1.4 Создание связи на фронтенде

Когда пользователь тянет от одного handle к другому:

1. Срабатывает **onConnect()** в flowStore.ts (строка ~568)
2. **isValidConnection()** в reactflowUtils.ts (строка ~370) проверяет совместимость типов:
– inputTypes таргета должны пересекаться с output_types источника
– у таргета еще нет связи (если не list и не loop)
– связь не создает цикл (если не loop-компонент)
3. Создается edge с сериализованными handle в строках sourceHandle/targetHandle и объектах data.sourceHandle/data.targetHandle
4. Edge добавляется в состояние React Flow

6.5.1.5 1.5 Последовательность загрузки flow

Когда пользователь открывает flow, происходит следующее:

```
1. GET /api/v1/flows/{flow_id}
   Бэкенд читает Flow из БД, возвращает сырой JSON без обработки
   Файл: src/backend/base/langflow/api/v1/flows.py:280

2. Фронтенд useGetFlow() получает ответ, вызывает processFlows()
   Файл: src/frontend/src/controllers/API/queries/flows/use-get-flow.ts:27

3. processFlows() вызывает processDataFromFlow()
   Файл: src/frontend/src/utils/reactflowUtils.ts:508-534

4. processDataFromFlow() запускает по порядку:
   a. processFlowEdges(flow)  - миграция старых форматов handle
   b. processFlowNodes(flow)  - миграция старых форматов output
   c. updateEdges()           - добавление анимации
   d. updateIds()             - обновление ID нод
   Файл: src/frontend/src/utils/reactflowUtils.ts:540-560

5. setCurrentFlow() вызывает resetFlow()
   Файл: src/frontend/src/stores/flowsManagerStore.ts:37-43

6. resetFlow() запускает:
   a. detectBrokenEdgesEdges() - генерирует алерты о сломанных связях
   b. cleanEdges()              - валидирует ВСЕ edge-и и удаляет невалидные
   c. updateComponentsToUpdate() - определяет устаревшие компоненты
   d. Записывает очищенные edges + nodes в Zustand store
   Файл: src/frontend/src/stores/flowStore.ts:212-246

7. React рендерит канвас с очищенными edge-ами
```

Важно: бэкенд НЕ делает никакой валидации при отдаче flow. Он возвращает сырой JSON из базы. Вся валидация связей происходит на фронтенде.

6.5.1.6 1.6 Валидация на фронтенде - cleanEdges()

Файл: src/frontend/src/utils/reactflowUtils.ts:76-177

Это основная функция валидации связей. Для каждого edge она делает:

Проверка target handle (строки 91-134):
1. Парсит сохраненную строку targetHandle обратно в объект
2. Ищет fieldName в текущем template целевой ноды
3. Читает текущие type и inputTypes из поля template
4. Собирает "ожидаемый" handle ID из текущего состояния ноды
5. Сериализует его через scapedJSONStringfy()
6. **Сравнивает строки - если отличаются, edge удаляется**

Проверка source handle (строки 136-171):
1. Парсит сохраненную строку sourceHandle
2. Ищет output по selected_output или по имени в sourceNode.data.node.outputs
3. Читает текущие types из найденного output
4. Собирает "ожидаемый" source handle ID
5. **Сравнивает строки - если отличаются или output не найден, edge удаляется**

Проверка скрытых полей (строка 174, определение строки 218-236):

- Если у поля show === false, edge удаляется

Проверка tool_mode (строка 131):

- Если у ноды tool_mode и поле isToolMode, edge удаляется независимо от совпадения строк

Удаление advanced полей (строки 179-211):

- Отдельная функция clearHandlesFromAdvancedFields() удаляет edge-и к полям с advanced === true
- Вызывается при изменении данных компонента, не при загрузке flow

6.5.1.7 1.7 Валидация на бэкенде

При сборке/запуске flow бэкенд создает объект Graph (graph/graph/base.py) и для каждой связи создает объект Edge.

Файл: src/backend/base/langflow/graph/edge/base.py:84-109

```
def _validate_handles(self, source, target):
    if self.target_handle.input_types is None:
        self.valid_handles = self.target_handle.type in
self.source_handle.output_types
    elif self.target_handle.type is None:
        # Loop edge - проверка пересечения output_types и input_types
        self.valid_handles = bool(self.source_handle.output_types) and (
            not self.target_handle.input_types
            or any(t in self.target_handle.input_types for t in
self.source_handle.output_types)
        )
```

```
    elif self.source_handle.output_types is not None:
        self.valid_handles = (
            any(t in self.target_handle.input_types for t in
self.source_handle.output_types)
            or self.target_handle.type in self.source_handle.output_types
        )
    if not self.valid_handles:
        raise ValueError(...)
```

Бэкенд валидирует по текущим определениям компонентов из кода, а не по сохраненным в flow данным. Если типы изменились - ValueError и flow не запускается.

6.5.1.8 1.8 Система шаблонов и определение устаревших компонентов

При старте приложения фронтенд загружает все определения компонентов с бэкенда и кладет в useTypesStore:

Файл: src/frontend/src/stores/typesStore.ts

```
setTypes: (data) => {
    set((old) => ({
        types: typesGenerator(data),
        data: { ...old.data, ...data },
        ComponentFields: extractFieldsFromComponenents({ ...old.data, ...data }),
        templates: templatesGenerator(data),
    }));
}
```

Поле templates содержит "эталонные" определения компонентов из текущей версии Langflow.

Файл: src/frontend/src/CustomNodes/helpers/check-code-validity.ts:53-81

checkCodeValidity() сравнивает:

- Код компонента в ноде vs код в эталонном шаблоне
- Если код отличается - компонент "устаревший"
- Если изменились outputs, ключи template или input_types - это "breaking change"

Эта проверка кормит UI-уведомление "Updates available".

6.5.2 2. Где ломается при обновлении

6.5.2.1 2.1 Сценарий обновления

1. Клиент работает на Langflow v1, flow-ы сохранены в базе
2. Деплоится новый Docker-образ с Langflow v2
3. Бэкенд начинает отдавать шаблоны компонентов v2 (новый код, возможно другие inputs/outputs/types)
4. Пользователь открывает свой flow
5. Фронтенд получает старые ноды из БД, но useTypesStore уже имеет шаблоны v2

6. cleanEdges() запускается на старых данных нод - edge-и удаляются
7. Auto-save сохраняет flow без edge-ей обратно в базу

6.5.2.2 2.2 Точка отказа 1 - строковое сравнение в cleanEdges()

Файл: src/frontend/src/utils/reactflowUtils.ts:129-134, 165-166

cleanEdges() собирает "ожидаемый" handle ID из текущего состояния ноды и сравнивает посимвольно с сохраненной строкой:

```
if (scapedJSONStringfy(id) !== targetHandle) {
    newEdges = newEdges.filter((e) => e.id !== edge.id);  // edge удален
}
```

Сравнение ломается когда **любое** свойство в handle изменилось, даже если связь все еще валидна по типам:

- input_types массив получил новый элемент или потерял старый
- type поля переименовано (например "str" на "text")
- output_types изменили порядок или содержимое
- fieldName переименовано в новой версии компонента

6.5.2.3 2.3 Точка отказа 2 - output не найден по имени

Файл: src/frontend/src/utils/reactflowUtils.ts:140-170

Для source handle cleanEdges() ищет output в sourceNode.data.node.outputs по имени:

```
const output =
    sourceNode.data.node!.outputs?.find(
        (output) => output.name === sourceNode.data.selected_output,
    ) ??
    sourceNode.data.node!.outputs?.find(
        (output) =>
            (output.selected || ...) && output.name === name,
    );
```

Если output переименовали между версиями, или selected_output ссылается на удаленный output - совпадение не найдено и **все edge-и от этого компонента удаляются** (строка 169).

6.5.2.4 2.4 Точка отказа 3 - несовпадение output_types в сериализованном handle

Сохраненный edge кодирует output_types на момент создания связи:

```
{"dataType": "ChatOpenAI", "id": "abc123", "output_types": ["Message"], "name":
"response"}
```

Если в новой версии тип выхода изменился с "Message" (see page 70) на "Data" (see page 70), собранный handle ID будет содержать "Data" (see page 70), а сохраненный edge все еще "Message" (see page 70) - строки не совпадают - edge удален.

6.5.2.5 2.5 Точка отказа 4 - скрытые поля удаляют edge-и

Файл: src/frontend/src/utils/reactflowUtils.ts:218-236

```
if (nodeTemplates[fieldName]?.show === false) {
    newEdges = newEdges.filter((e) => e.id !== edge.id);
}
```

Если обновление шаблона меняет show на false (поле стало внутренним/advanced), edge-и к этому полю удаляются - хотя поле все еще существует.

6.5.2.6 2.6 Точка отказа 5 - валидация на бэкенде при запуске

Файл: src/backend/base/langflow/graph/edge/base.py:84-109

Даже если edge-и выжили на фронтенде, бэкенд валидирует их по текущим определениям компонентов при сборке графа. Несовпадение типов - ValueError и flow не работает.

6.5.2.7 2.7 Почему это тихо

- cleanEdges() удаляет edge-и без уведомления пользователя
- detectBrokenEdgesEdges() запускается до cleanEdges() и показывает UI-алерты через useAlertStore, но у нее другая логика поиска source output (см. Баг 4) - поэтому cleanEdges() может удалить edge-и, которые detectBrokenEdgesEdges() не заметила
- Проверка tool_mode в cleanEdges() вообще не имеет аналога в detectBrokenEdgesEdges()
- Пользователь просто открывает flow и видит что связей нет

6.5.2.8 2.8 Почему это необратимо

После того как resetFlow() удалил edge-и, auto-save записывает flow без связей обратно в базу. Оригинальный flow с edge-ами потерян. Восстановить можно только из бэкапа базы.

6.5.3 3. Найденные баги

6.5.3.1 Баг 1: cleanEdges() сравнивает строки вместо проверки совместимости типов

Файлы: src/frontend/src/utils/reactflowUtils.ts:129-134, 165-166

Проблема: Функция сериализует "ожидаемый" handle из текущего состояния ноды и сравнивает строку с сохраненной. Любое изменение inputTypes, type или output_types ломает совпадение, даже если связь все еще валидна по типам.

Пример: Новая версия добавляет тип в input_types ("Message" (see page 70) на "Message", "Data" (see page 70)). Связь все еще валидна - Message как принимался, так и принимается. Но строки разные - edge удален.

Корневая причина: Используется строковое сравнение (scapedJSONStringfy(id) !== targetHandle) вместо проверки пересечения типов.

Направление фикса: Заменить строковое сравнение на семантическую валидацию - проверять что output_types источника пересекаются с input_types таргета.

6.5.3.2 Баг 2: шаблоны не обновляются до очистки edge-ей

Файлы: src/frontend/src/stores/flowStore.ts:212-246

Проблема: Последовательность при загрузке flow:
1. Загрузить старые ноды из БД (с устаревшими данными шаблона)
2. Запустить cleanEdges() на этих старых нодах
3. Запустить updateComponentsToUpdate() для определения устаревших компонентов

Но cleanEdges() валидирует edge-и по сохраненным данным нод, в которых старые input_types, output_types, type. Ноды еще не согласованы с новыми шаблонами. При этом processFlowNodes/processFlowEdges могли частично обновить данные, создавая рассогласование между handle в edge-ах и данными в нодах.

Корневая причина: Очистка edge-ей запускается до согласования шаблонов. updateComponentsToUpdate() (который определяет изменения) выполняется после того как cleanEdges() уже удалил edge-и.

Направление фикса: Либо согласовывать шаблоны нод до очистки edge-ей, либо откладывать очистку до момента когда пользователь явно обновит компоненты.

6.5.3.3 Баг 3: auto-save делает потерю edge-ей необратимой

Файлы: src/frontend/src/stores/flowStore.ts:212-246, механизм auto-save

Проблема: После того как resetFlow() удалил edge-и, очищенное состояние flow записывается в Zustand store. Auto-save затем сохраняет это состояние в базу. Оригинальный flow с edge-ами потерян навсегда.

Влияние: Пользователи не могут восстановить связи без бэкапа базы.

Направление фикса: Не запускать auto-save сразу после загрузки flow. Или: сохранять оригинальные данные flow до очистки и дать возможность восстановления. Или: спрашивать пользователя перед удалением edge-ей и сохранением.

6.5.3.4 Баг 4: detectBrokenEdgesEdges() и cleanEdges() по-разному ищут source output

Файлы: src/frontend/src/utils/reactflowUtils.ts:238+ vs src/frontend/src/utils/reactflowUtils.ts:76-177

Проблема: Обе функции используют одинаковое строковое сравнение для target handle. Но для source handle они работают по-разному:

- cleanEdges() (строки 141-152) сначала ищет по selected_output, потом fallback по name с доп. условиями (output.selected или единственный не-grouped output)
- detectBrokenEdgesEdges() (строки 334-336) просто ищет по name: outputs?.find((output) => output.name === name)

Плюс cleanEdges() имеет проверку tool_mode (строка 131), которая удаляет edge-и независимо от совпадения строк - а в detectBrokenEdgesEdges() такой проверки нет.

Из-за этого edge-и могут быть удалены cleanEdges() без появления в алерте - конкретно когда:

- selected_output находит другой output, чем detectBrokenEdgesEdges() находит по name
- tool_mode удаляет edge, а detectBrokenEdgesEdges() это не ловит

Направление фикса: Вынести логику валидации в одну общую функцию, чтобы все edge-и, которые cleanEdges() удалит, сначала попадали в алерт.

6.5.3.5 Баг 5: selected_output ссылается на удаленный output

Файлы: src/frontend/src/utils/reactflowUtils.ts:141-152, src/backend/base/langflow/custom/custom_component/component.py

Проблема: Если outputs компонента изменились между версиями (output переименован или удален), а в сохраненной ноде selected_output все еще указывает на старое имя - поиск в cleanEdges() не находит output:

```
const output = sourceNode.data.node!.outputs?.find(
    (output) => output.name === sourceNode.data.selected_output,  // старое имя - не
найдено
) ?? sourceNode.data.node!.outputs?.find(/* fallback тоже не сработает */);
```

Когда output не найден - **все edge-и от этого компонента удаляются** (строка 169).

На бэкенде component.py тоже валидирует selected_output против _outputs_map и кидает ValueError если output не существует.

Направление фикса: Когда selected_output не найден - фолбэчить на первый доступный output вместо удаления всех edge-ей. Или: мигрировать selected_output при согласовании шаблонов.

6.6 Компоненты LLMOps

6.7 Описание требований и архитектуры

6.7.1 Копия Stack [AS IS]

Дата актуализации	**25.02.25**

6.7.1.1 Текущий стек реализуется компонентами:

Components	**Functionality**	**License**	**Link**
LiteLLM	LLM and AI Tools	License-1.0	https://github.com/BerriAI/litellm?tab=License-1-ov-file
LibreChat	LLM and AI Tools	MIT	https://github.com/danny-avila/LibreChat?tab=MIT-1-ov-file
Langflow	LLM and AI Tools	MIT	https://github.com/langflow-ai/langflow
PostgreSQL	Databases and Caching	PostgreSQL License	https://opensource.org/license/postgresql
Patroni	Databases and Caching	MIT	https://github.com/patroni/patroni?tab=MIT-1-ov-file
KeyDB	Databases and Caching	BSD-3-Clause	https://github.com/Snapchat/KeyDB?tab=BSD-3-Clause-1-ov-file
MongoDB	Databases and Caching	Server Side Public License (SSPL)	https://www.mongodb.com/legal/licensing/server-side-public-license
OpenSearch	Databases and Caching	Apache-2.0	https://github.com/opensearch-project/OpenSearch?tab=Apache-2.0-1-ov-file

Components	Functionality	License	Link
VictoriaMetrics	Monitoring and Logging	Apache-2.0	https://github.com/VictoriaMetrics/VictoriaMetrics?tab=Apache-2.0-1-ov-file
Grafana	Monitoring and Logging	AGPL-3.0	https://github.com/grafana/grafana?tab=AGPL-3.0-1-ov-file
Vector	Monitoring and Logging	Apache-2.0	https://www.mozilla.org/en-US/MPL/2.0/
Alertmanager	Monitoring and Logging	Apache-2.0	https://github.com/prometheus/alertmanager?tab=Apache-2.0-1-ov-file
Blackbox_exporter	Monitoring and Logging	Apache-2.0	https://github.com/prometheus/blackbox_exporter?tab=Apache-2.0-1-ov-file
cAdvisor	Monitoring and Logging	License-1.0	https://github.com/google/cadvisor?tab=License-1-ov-file
Haproxy	Load Balancing and Proxying	License-1.0	https://github.com/haproxy/haproxy?tab=License-1-ov-file
Nginx	Load Balancing and Proxying	BSD-2-Clause	https://github.com/nginx/nginx?tab=BSD-2-Clause-1-ov-file
NSD	Networking and DNS Services	BSD-3-Clause	https://github.com/NLnetLabs/nsd/blob/master/LICENSE
Unbound	Networking and DNS Services	BSD-3-Clause	https://github.com/NLnetLabs/unbound/blob/master/LICENSE
Docker	Orchestration and Virtualization	Apache-2.0	https://github.com/docker/docker-install?tab=Apache-2.0-1-ov-file
Docker-compose	Orchestration and Virtualization	Apache-2.0	https://github.com/docker/compose?tab=Apache-2.0-1-ov-file

Components	Functionality	License	Link
Pyinfra	Infrastructure automation and server configuration management using Python	MIT	https://github.com/pyinfra-dev/pyinfra
Proxmox	Orchestration and Virtualization	AGPL-3.0	https://www.gnu.org/licenses/agpl-3.0.html
Consul	Configuration Management and High Availability	License-1.0	https://github.com/hashicorp/consul?tab=License-1-ov-file
Etcd	Configuration Management and High Availability	Apache-2.0	https://github.com/etcd-io/etcd?tab=Apache-2.0-1-ov-file
Chrony	Configuration Management and High Availability	License-1.0	https://github.com/mlcivhar/chrony?tab=License-1-ov-file
Chrony_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/SuperQ/chrony_exporter?tab=Apache-2.0-1-ov-file
Haproxy_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/prometheus/haproxy_exporter?tab=Apache-2.0-1-ov-file
Redis_exporter	Prometheus Exporters for Metrics	MIT	https://github.com/oliver006/redis_exporter?tab=MIT-1-ov-file
Nginx_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/nginxinc/nginx-prometheus-exporter?tab=Apache-2.0-1-ov-file
Node_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/prometheus/node_exporter?tab=Apache-2.0-1-ov-file
NSD_exporter	Prometheus Exporters for Metrics	BSD-3-Clause	https://github.com/optix2000/nsd_exporter?tab=BSD-3-Clause-1-ov-file

Components	Functionality	License	Link
MongoDB_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/percona/mongodb_exporter?tab=Apache-2.0-1-ov-file
Nvidia_gpu_exporter	Prometheus Exporters for Metrics	MIT	https://github.com/utkuozdemir/nvidia_gpu_exporter?tab=MIT-1-ov-file
Postgres_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/prometheus-community/postgres_exporter?tab=Apache-2.0-1-ov-file
Unbound_exporter	Prometheus Exporters for Metrics	Apache-2.0	https://github.com/letsencrypt/unbound_exporter?tab=Apache-2.0-1-ov-file

6.7.2 Сценарии использования

6.7.2.1 Сценарии использования LibreChat [UI]

6.7.2.1.1 [Librechat.UC.01] Выбор моделей и управление параметрами - разделить!

6.7.2.1.1.1 Введение

Контекст	Платформа использует различные модели для генерации ответов. Ранее в проекте возникла необходимость разграничить **«модели» и «агентов» (aka ассистенты)** • **Модели** – сущности, отвечающие только за генерацию ответа. • **Агенты (aka ассистенты)** – более высокоуровневые абстракции, в которых параметры инференса зашиты хардкодом, то есть являются дефолтными. По мере развития платформы стало понятно, что конечным пользователям необходимо давать возможность **гибко управлять** такими параметрами, как температура, максимальная длина ответа, настройки «штрафов» и т. д
Цель	Предоставить пользователю **управляемый и прозрачный** механизм выбора модели и настройки параметров инференса, при этом избежать конфликтов между существующими «агентами» и «моделями», а также унифицировать логику пресетов.

6.7.2.1.1.2 **Описание проблематики**

Текущее состояние	**Модели**: • Добавляются на платформу отдельно. • Пресеты привязаны к модели, что позволяет сохранять некоторые настройки, однако не все параметры учитываются. **Агенты (ассистенты)**: • Параметры инференса (температура, penalti и др.) жёстко зашиты, чтобы упростить использование для конечного пользователя. • При попытке пользователя менять параметры напрямую происходят конфликты, поскольку пресеты не ожидают работу через «агента», а не напрямую через «модель». **LibreChat**: • Предусматривает работу с пресетами, опирающимися на понятие «модель». • При создании пресета для «агента» возникают проблемы, так как агент не ведёт себя как модель в архитектуре системы.
Примеры	
Критичность	• Уменьшает удобство для продвинутых пользователей, которым нужна гибкая настройка инференса (например, для экспериментов). • Снижает общее качество ответа моделей (при неправильных параметрах).

6.7.2.1.2 [Librechat.UC.02] Создание и использование пресета в LibreChat для регулярных типовых запросов

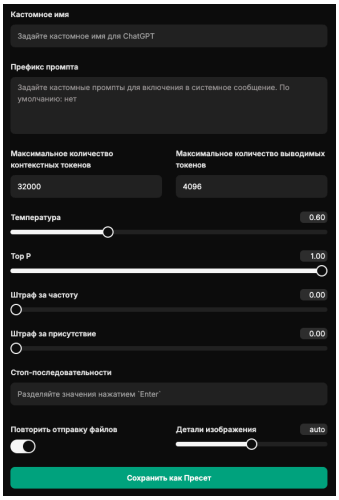
Цель использования	Упростить и ускорить регулярное взаимодействие с LibreChat, минимизируя повторяющиеся настройки и повышая стабильность результатов
Контекст	Пользователь регулярно обращается к LibreChat для выполнения типовых профессиональных задач. Для повышения качества и единообразия ответов он применяет заранее настроенный пресет

Пользовательские проблемы до использования пресета	• Регулярные запросы требуют повторного ручного ввода похожих настроек • Ответы часто отличаются по структуре или стилю • Отсутствует стандартизация при регулярном взаимодействии с моделью
Ценность для пользователя после применения пресета	• **Экономия времени:** один раз настроив пресет, пользователь получает быстрый доступ к повторяющимся типовым запросам • **Консистентность результатов:** пресет обеспечивает стабильность формата, стиля и содержания ответов • **Упрощение взаимодействия:** минимизация ручных операций при регулярном использовании

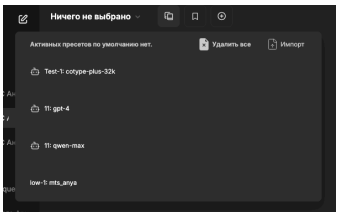
6.7.2.1.2.1 Сценарий взаимодействия

AS IS [v.0.1] AS IS [v.0.1] - Schema

	Действие пользователя	**Интерфейс**
Описание	Пользователь регулярно обращается с типовыми запросами к LibreChat и хочет оптимизировать процесс взаимодействия, сократить время на ввод текста и обеспечить унификацию запросов за счёт заранее подготовленных шаблонов (пресетов)	
Участники	• **Пользователь** — конечный пользователь системы LibreChat • **LibreChat** — система интеллектуального помощника с интерфейсом для настройки и взаимодействия через пресеты	
Триггер	Пользователь решил создать новый или использовать уже существующий пресет для отправки регулярного типового запроса	
Предусловия	• Пользователь авторизован в системе LibreChat. • У пользователя есть доступ к функции создания и управления пресетами • Интерфейс LibreChat поддерживает работу с пресетами	

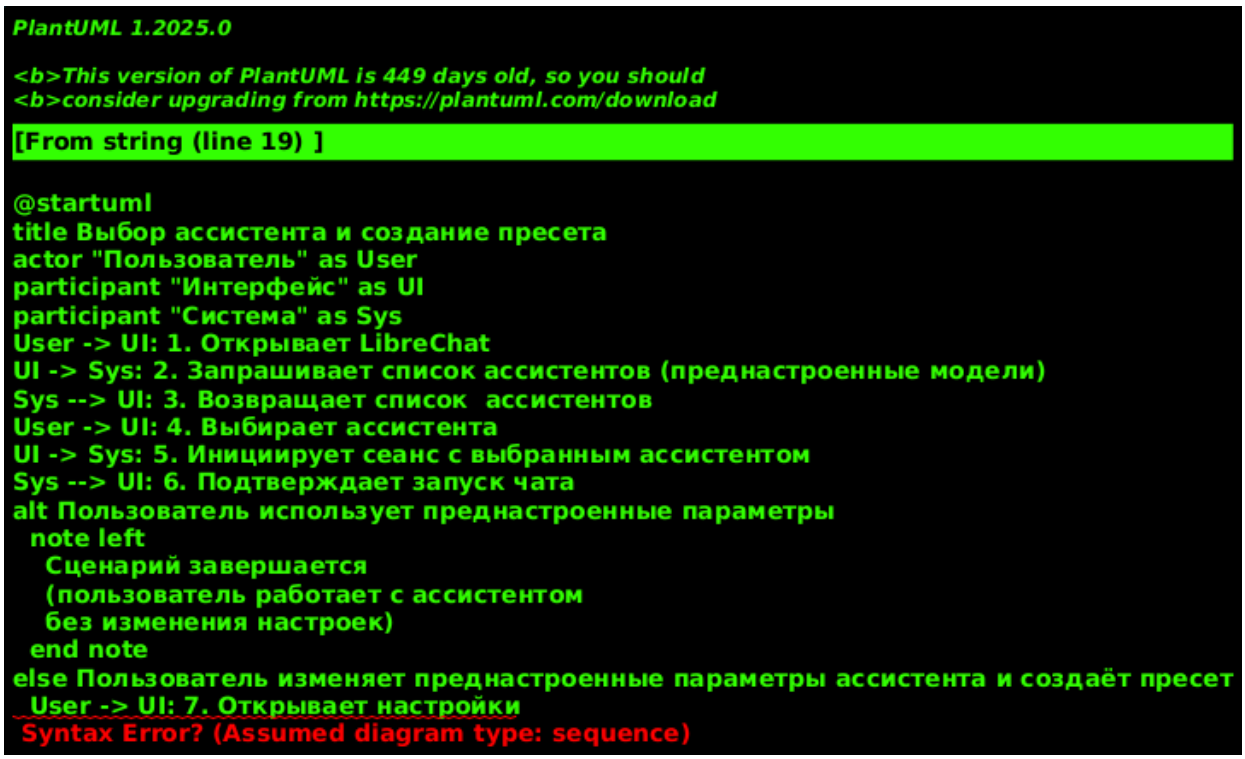
Постусловия	• Запрос отправлен с использованием пресета и обработан системой • Созданный пресет сохранен в системе LibreChat и доступен для повторного использования	
Успешный сценарий	• Пользователь выбирает функцию «Создание нового пресета» в интерфейсе LibreChat • LibreChat предлагает пользователю форму для создания нового пресета, включая поля «Название», «Описание» (необязательно), «Текст запроса» и «Параметры запроса» • Пользователь заполняет необходимые поля и сохраняет пресет • LibreChat подтверждает успешное сохранение пресета. • Для последующего использования пресета пользователь выбирает функцию «Использование существующего пресета» • Пользователь выбирает из списка нужный пресет и нажимает «Отправить запрос» • LibreChat автоматически подставляет текст и параметры пресета в окно ввода и отправляет запрос • LibreChat отображает полученный от системы ответ пользователю	
Альтернативный сценарий 1 Редактирование существующего пресета	1. Пользователь выбирает функцию «Редактирование пресетов» 2. Пользователь выбирает из списка пресет, который необходимо отредактировать 3. LibreChat отображает текущие значения пресета и предоставляет возможность внести изменения 4. Пользователь редактирует текст и параметры пресета и сохраняет изменения 5. LibreChat подтверждает успешное сохранение изменений	

Альтернативный сценарий 2 Удаление пресета	1. Пользователь выбирает функцию «Редактирование пресетов» 2. Пользователь выбирает пресет для удаления 3. LibreChat запрашивает подтверждение удаления пресета 4. Пользователь подтверждает удаление 5. LibreChat удаляет пресет из системы и отображает сообщение об успешном удалении	

UI-Описание	UI-Описание: Интерфейс системы LibreChat должен содержать следующие элементы для работы с пресетами: • **Кастомное имя** (необязательное текстовое поле) – позволяет задать уникальное имя пресета для идентификации в интерфейсе • **Префикс промпта** (необязательное многострочное текстовое поле) – предназначен для задания шаблонных инструкций, которые будут автоматически включаться в системные сообщения перед запросами • **Максимальное количество контекстных токенов** (числовое поле) – позволяет ограничивать количество токенов, учитываемых системой при формировании контекста ответа • **Максимальное количество выводимых токенов** (числовое поле) – задаёт ограничение для длины ответов системы • **Температура** (ползунок или числовое поле) – позволяет регулировать степень случайности (креативности) ответов • **Top P** (ползунок или числовое поле) – регулирует диапазон вероятностей слов при генерации текста • **Штраф за частоту** (ползунок или числовое поле) – уменьшает вероятность повторения часто используемых слов • **Штраф за присутствие** (ползунок или числовое поле) – уменьшает вероятность повторения уже использованных слов • **Стоп-последовательности** (необязательное текстовое поле) — возможность задавать специальные фразы или символы, после которых генерация текста должна быть прекращена • **Повторить отправку файлов** (переключатель) – включает или отключает автоматическую повторную отправку всех ранеее отправленных файлов при использовании пресета • **Детали изображения** (ползунок) – позволяет настроить степень детализации при обработке изображений. Низкое, высокое или авто-режим Управление и использование пресетов: • Меню выбора пресета – позволяет выбирать ранее сохранённые пресеты для быстрого использования • Кнопка «Удалить все» – позволяет удалить все ранее созданные пресеты	

	• Кнопка «Импорт» – позволяет импортировать пресеты из внешнего источника	

AS IS [v.0.1] AS IS [v.0.1] - Schema



6.7.2.1.2.2 Выявленная проблематика и решение

Описание проблемы и решения Schema

Предыстория и постановка задачи	В ходе эксплуатации Librechat (агент) была обнаружена серия проблем, связанных с: 1. Сбросом промпта при использовании агента. 2. Потерей/некорректной передачей пресета конфигурации между клиентской и серверной частью. После обновления компонента до версии `0.7.7` часть сбоев пропала, но проблему с передачей пресета по-прежнему требовалось решать системно.

Суть проблемы	• **Сброс промпта**: ранее наблюдалось, что при использования агента или при частом переключении параметров, данные (промпт) могли сбрасываться. Как выяснилось, часть этих багов была устранена при обновлении до версии `0.7.7`. • **Неправильное использование пресета**: в ряде случаев возникала ситуация, когда настройки пользователя, хранящиеся в пресете, терялись либо не применялись при повторных обращениях со стороны UI. Оказалось, что ключевые параметры и идентификатор пресета не всегда корректно передавались в серверную часть и обратно.
Решение	1. **Обновление компонента**: • Обновлённая версия `0.7.7` устранила несколько внутренних ошибок, связанных со сбросом промпта 2. **Передача идентификатора пресета**: • Для решения проблемы с пресетом был реализован механизм, при котором **идентификатор (ID) пресета** всегда явно передаётся от клиентской части на сервер и затем сервер возвращает этот же идентификатор обратно • Такой подход позволил *однозначно* идентифицировать настройки, хранящиеся на сервере, и повторно их применять в нужном контексте

Технические детали реализации	**Расширенный формат запроса от UI**:
	• Вместо передачи отдельных параметров (без связи друг с другом) теперь в запросе от UI всегда присутствует поле `preset_id` вместе со всеми необходимыми атрибутами.
	• Пример структуры:
	`{ "preset_id": "some-unique-id", "params": { "paramA": "valueA", "paramB": "valueB" } }`
	
	Обработка на стороне бэкенда:
	• При получении запроса backend извлекает `preset_id` и находит соответствующие настройки (если они уже сохранены)
	• После выполнения необходимых операций (валидация, применение логики, генерация ответа) backend возвращает в ответе этот же `preset_id` вместе с результирующими данными
	• Обратный ответ может выглядеть так:
	`{ "preset_id": "some-unique-id", "result": { "data": "some-calculated-result" } }`

```
}
```

Таким образом, UI всегда «узнаёт» результат, связанный именно с тем пресетом, который был отправлен

- **Возможные сценарии использования**:
  - **Создание нового пресета**: UI генерирует новый `preset_id` и отправляет набор параметров на сервер, получая назад подтверждение и итоговые данные
  - **Обновление существующего пресета**: UI отправляет тот же `preset_id`, а сервер модифицирует нужные параметры или сохраняет состояние
  - **Переключение между пресетами**: UI может хранить несколько `preset_id` и в любой момент «активировать» нужный, гарантируя целостность конфигурации

+ UPD

Глобальные изменения:
- Добавлены пресеты для моделей при описании ендпоинтов
- Добавлены кастомные поля для моделей, используются только в UI:
  - label - отображаемое название (строка)
  - iconURL - ссылка на иконку (строка)
  - isOutside - внешняя или внутренняя модель (boolean)
- Добавлен атрибут default для endpoint (boolean). Обязательно к заполнению, если скрыто меню ендпоинтов, иначе по умолчанию будет ендпоинт openAI

UI и UX изменения:
- Кастомное отображение моделей в меню моделей (В шапке, в настройках и в редакторе промптов)
- Кнопка сброса пресета
- Изменен iconURL, добавлена поддержка иконок из папки /assets/ (актуальный список иконок здесь - https://gitlab.services.mts.ru/LLLMOps/frontend/librechat/-/tree/master/client/public/assets/models). Для добавления можно использовать оносительный путь: iconURL: /assets/models/mts.svg

Новый формат librechat.yaml

```
endpoints:
  custom:
    - name: 'Models' # (строка как и было)
      default: true # (новый параметр испольуется
для выбора ендпоинта по умолчанию)
      apiKey: '12343455' # (строка, как и было)
      baseURL: 'http://api.llmops.mts-corp.ru/v1' #
(строка, как и было)
      models:
        default: ["gpt-4", "qwen-coder-7b", "gpt-4-
turbo", "claude-3-haiku", "deepseek-coder",
"claude-3-opus", "gpt-4o", "gpt-3.5-turbo",
"claude-3-sonnet", "gpt-4o-mini", "gemini-pro",
"mts_anya", "kodify-1.0", "cotype-light-3.0", "qwen-
max", "qwen-turbo", "qwen-plus", "qwen2-7b-instruct",
 "qwen2.5-vl-7b-instruct"]
        # fetch: true
        fetch: false # (конифиги работают только для
описанных моделей, поэтому лучше отключить опцию
загрузки моделей динамически и обновлять ручками)

      modelsOptions: # (новый параметр, модели
описываются как объект)
        # MTS
        mts_anya:
          label: "MTS ANYA" # (нащвание,
отображается в меню и как название модели в чате)
          iconURL: "/assets/models/mts.svg" #
(ссылка на иконку модели, поддерживается
относительный путь)
          isOutside: false # (показывает внешняя
модель или внутренняя)
          preset: # (параметры пресета, пока
поддерживаются только эти 3)
            maxContextTokens: "128000"
            max_tokens: "4096"
            temperature: 0.3

        kodify-2.0:
          label: "Kodify 2.0"
          iconURL: "/assets/models/mts.svg"
          isOutside: false
          preset:
            maxContextTokens: "32000"
            max_tokens: "4096"
            temperature: 0.6
```

```
        cotype-light-3.0:
          label: "Cotype Light 3.0"
          iconURL: "/assets/models/mts.svg"
          preset:
            maxContextTokens: "128000"
            max_tokens: "4096"
            temperature: 0.6

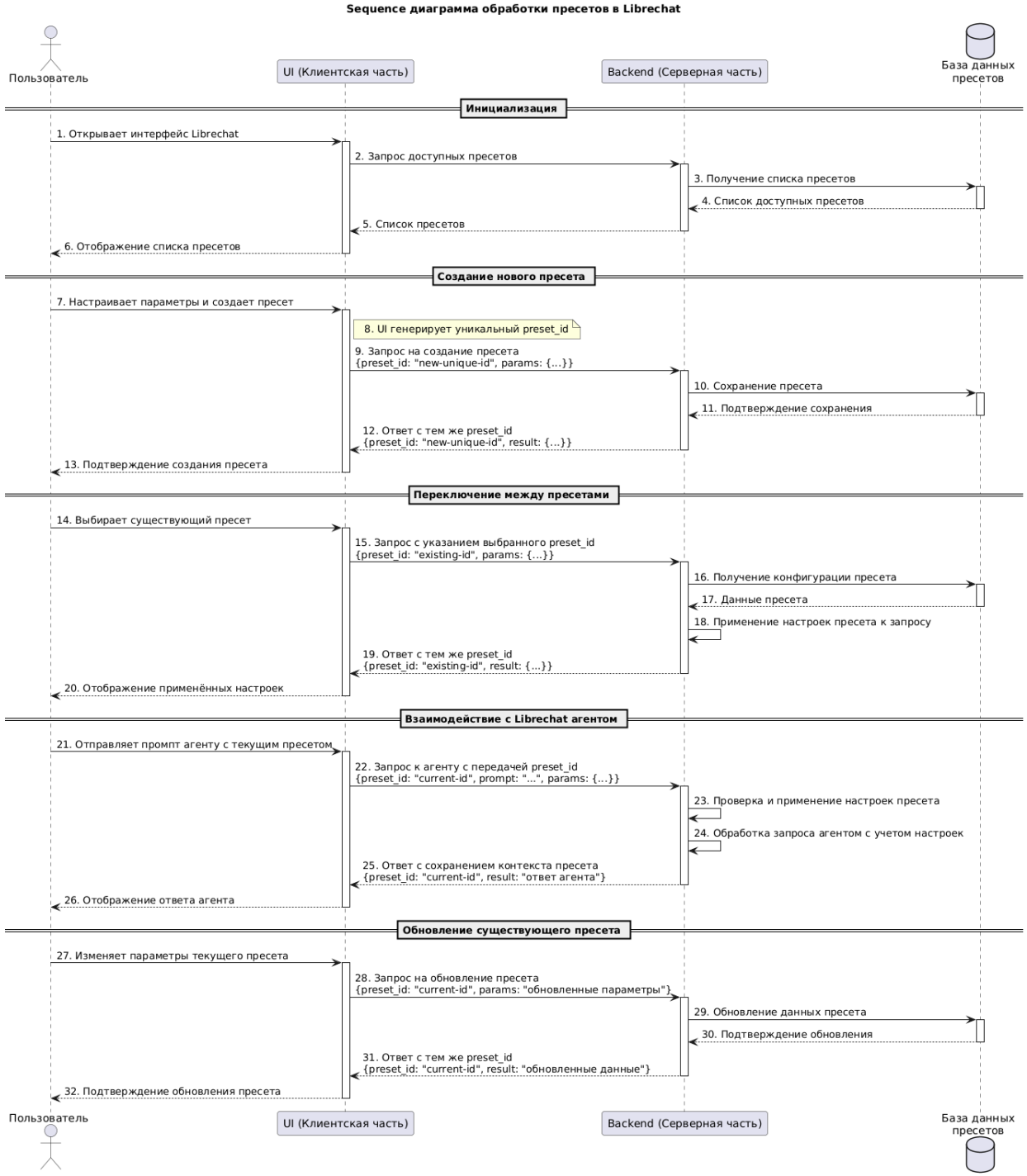
        cotype-plus-32k:
          label: "Cotype Plus 32k"
          iconURL: "/assets/models/mts.svg"
          preset:
            maxContextTokens: "32000"
            max_tokens: "4096"
            temperature: 0.6

        kodify-1.0:
          label: "Kodify 1.0 (CotypePro)"
          iconURL: "/assets/models/mts.svg"
          isOutside: false
          description: "kodify-1.0"
          preset:
            maxContextTokens: "128000"
            max_tokens: "4096"
            temperature: 0.6
```

+ UPD

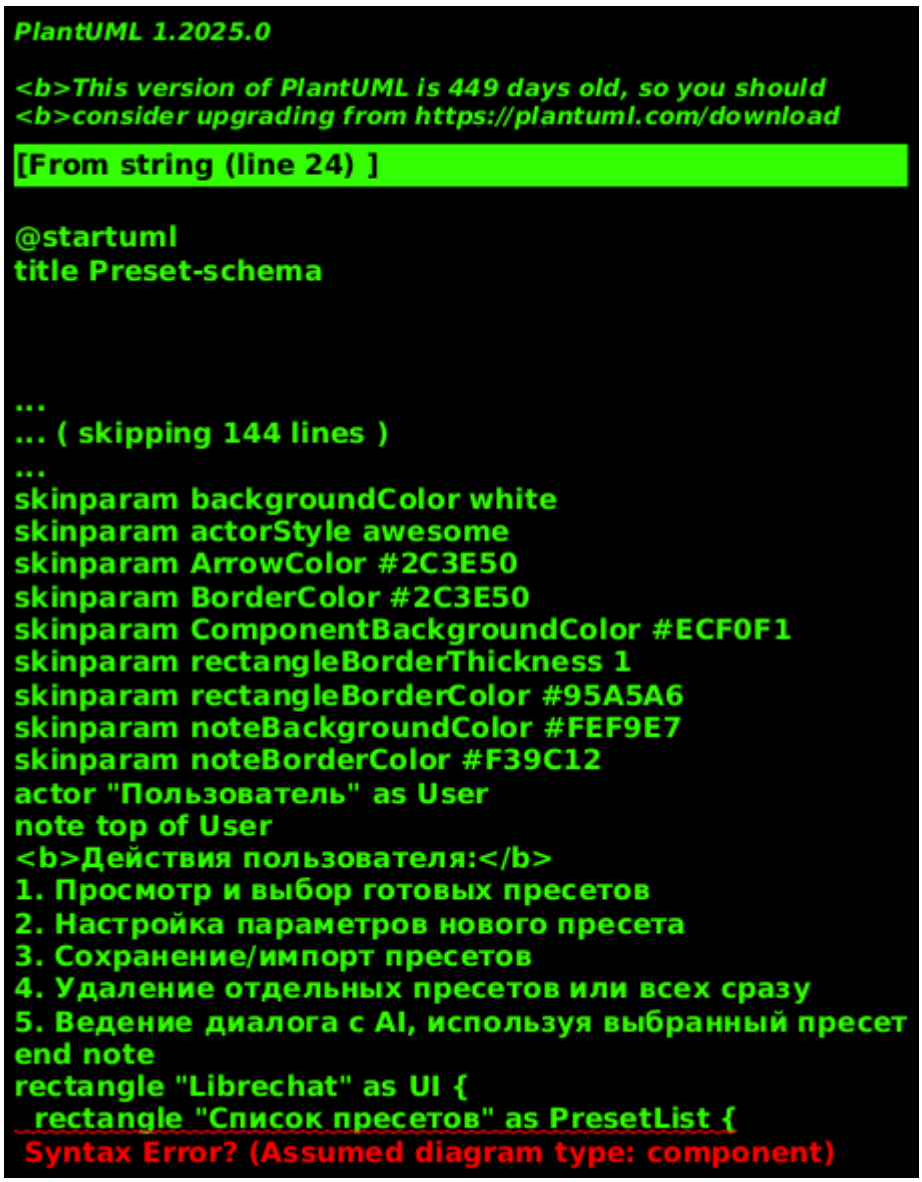
```
# показывать модалку (true/false, по умолчанию false)
SHOW_EULA_MODAL=
# подпись справа в подвале
FOOTER_LABEL=
# имя компании в подвале
FOOTER_COMPANY_NAME=
# url логотипа на шаге авторизации
HEADER_LOGO_URL=
# текст рядом с логотипом на шаге авторизации
HAEDER_LABEL=
```

Описание проблемы и решения | Schema



6.7.2.1.2.3 Librechat Schemas

Librechat preset schema



6.7.2.1.3 [Librechat.UC.03] Выбор модели (ассистента) из списка

Цель использования	Предоставить пользователю возможность выбора оптимальной модели для конкретного запроса и рабочего контекста

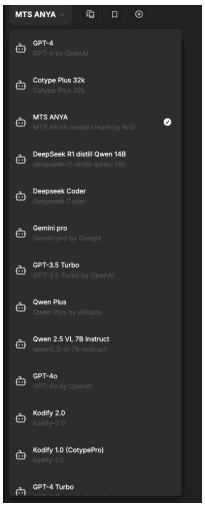
Контекст	Пользователь взаимодействует с LibreChat и нуждается в возможности быстро выбирать или менять используемую модель для конкретной задачи
Пользовательские проблемы до использования пресета	TBD
Ценность для пользователя после применения пресета	TBD

6.7.2.1.3.1 Сценарий взаимодействия

AS IS [v.0.1] TO BE []

	Действие пользователя	**Интерфейс**
Описание	Пользователь имеет возможность выбирать подходящую модель (ассистента) из предложенного списка в интерфейсе LibreChat для выполнения задач, соответствующих определённым требованиям по производительности, точности и специфике запросов	
Участники	• **Пользователь** — конечный пользователь, взаимодействующий с системой LibreChat • **LibreChat** — система интеллектуального помощника, предоставляющая интерфейс выбора модели из списка	
Триггер	Пользователь решил выбрать модель (ассистента) перед отправкой запроса, чтобы получить наиболее подходящий результат по качеству и специфике задачи	

Предусловия	• Пользователь авторизован и имеет доступ к функционалу выбора модели • Система LibreChat имеет в наличии список доступных моделей для выбора пользователем • Интерфейс LibreChat отображает актуальную информацию о доступных моделях	
Постусловия	• Пользователь успешно выбрал необходимую модель • Запросы пользователя направляются и обрабатываются выбранной моделью	
Успешный сценарий	1. Пользователь переходит в интерфейс выбора модели 2. LibreChat отображает список доступных моделей с указанием их названия и кратких характеристик 3. Пользователь изучает список моделей и выбирает подходящую модель 4. Пользователь подтверждает свой выбор (кликом на модель) 5. LibreChat сохраняет выбранную модель в текущей сессии (**УТОЧНИТЬ ПРО АПСТРИМЫ)** 6. Последующие запросы пользователя автоматически направляются выбранной модели 7. LibreChat возвращает ответы, соответствующие выбранной модели	
Альтернативный сценарий 1 Изменение модели после первичного выбора	1. Пользователь решает изменить текущую выбранную модель 2. Пользователь снова переходит в меню выбора моделей 3. LibreChat отображает список доступных моделей с текущим активным выбором 4. Пользователь выбирает другую модель 5. LibreChat переключает активную модель на новую, информирует пользователя о смене модели 6. Дальнейшие запросы пользователя обрабатываются новой моделью	

UI-Описание	Интерфейс системы LibreChat должен обеспечивать следующие элементы для выбора модели: • **Меню или выпадающий список доступных моделей** (например, GPT-4, GPT-3.5 Turbo, Claude, Qwen, и т.д.) • Отображение названия и краткой информации (краткое описание) рядом с каждой моделью • Визуальное выделение активной (текущей) модели в списке • Возможность быстрого переключения модели непосредственно перед отправкой запроса или в процессе диалога без потери контекста	

AS IS [v.0.1] TO BE []

6.7.2.1.3.2 Выявленная проблематика

6.7.2.1.4 [Librechat.UC.04] Закладки (добавление/удаление чата)

Цель использования	Предоставить пользователю возможность выбора оптимальной модели для конкретного запроса и рабочего контекста
Контекст	Пользователь взаимодействует с LibreChat и нуждается в возможности быстро выбирать или менять используемую модель для конкретной задачи
Пользовательские проблемы до использования	TBD
Ценность для пользователя после применения	TBD

6.7.2.1.4.1 Сценарий взаимодействия

AS IS [v.0.1] TO BE []

	Действие пользователя	**Интерфейс**
Описание	Пользователь хочет сохранять важные или часто используемые диалоги (чаты) в виде закладок, чтобы быстро возвращаться к ним в будущем или удалять более ненужные закладки для эффективного управления информацией	
Участники	• **Пользователь** — конечный пользователь системы LibreChat • **LibreChat** — интеллектуальный помощник, предоставляющий функционал управления закладками чатов	
Триггер	Пользователь решил сохранить текущий чат в закладках для последующего быстрого доступа, или, наоборот, удалить чат из списка закладок	
Предусловия	• Пользователь авторизован в системе LibreChat • Пользователь ведёт или ранее вёл диалог, который хочет добавить или удалить из закладок	
Постусловия	• Чат успешно добавлен в закладки или удалён из закладок • Изменения сохранены системой LibreChat • Закладка доступна в списке/удалена из списка	
Успешный сценарий	1. Пользователь открывает чат, который хочет сохранить 2. Пользователь нажимает на иконку закладки («Добавить в закладки») 3. LibreChat подтверждает успешное добавление чата в закладки 4. Чат отображается в списке закладок и доступен для быстрого доступа	

Альтернативный сценарий Удаление чата из закладок	1. Пользователь переходит в меню или панель закладок 2. Пользователь выбирает чат, который необходимо удалить из закладок 3. Пользователь нажимает кнопку «Удалить из закладок» 4. LibreChat запрашивает подтверждение удаления чата из закладок 5. Пользователь подтверждает удаление 6. LibreChat удаляет чат из закладок и уведомляет пользователя об успешном выполнении операции	

UI-Описание	Интерфейс системы LibreChat для управления закладками должен содержать следующие элементы: • **Кнопка «Закладки»** — расположена на основной панели инструментов и открывает меню управления закладками. • Список сохранённых закладок — отображает ранее добавленные чаты с указанием их заголовков. • Кнопка «Очистить всё» — позволяет удалить сразу все сохранённые закладки. • **Форма добавления закладки**, содержащая следующие поля: • Заголовок (обязательное текстовое поле) — позволяет задать название закладки для удобной идентификации. • Описание (необязательное многострочное текстовое поле) — служит для добавления дополнительной информации о закладке. • Переключатель «Добавить в текущий чат» — при активации добавляет закладку непосредственно в текущий чат пользователя. • **Кнопки действий формы**: • «Сохранить» — для сохранения закладки в системе. • «Отмена» — для отмены создания или редактирования закладки. • **Меню управления закладками** с функциями: • Просмотр списка ранее созданных закладок. • Быстрый переход к сохранённым чатам. • Возможность удаления отдельных закладок. • Кнопка **«Очистить всё»** для удаления всех закладок одновременно с подтверждением действия во избежание случайных удалений.	

AS IS [v.0.1] TO BE []

6.7.2.1.4.2 Выявленная проблематика

6.7.2.1.5 [Librechat.UC.05] Отправка сообщения

Цель использования	Предоставить пользователю возможность выбора оптимальной модели для конкретного запроса и рабочего контекста
Контекст	Пользователь взаимодействует с LibreChat и нуждается в возможности быстро выбирать или менять используемую модель для конкретной задачи
Пользовательские проблемы до использования	TBD
Ценность для пользователя после применения	TBD

6.7.2.1.5.1 Сценарий взаимодействия

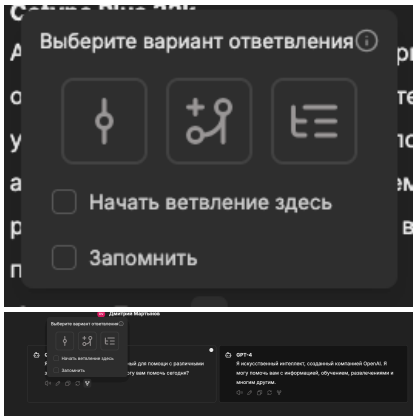
AS IS [v.0.1] TO BE []

	Действие пользователя	**Интерфейс**
Описание	Пользователь вводит сообщение в LibreChat и отправляет его на обработку. Система может направить запрос к одной выбранной модели ИИ или одновременно нескольким моделям. Если пользователь выбирает несколько моделей, он получает несколько ответов и может сравнить их, выбрав наиболее подходящий для дальнейшего диалога. Это обеспечивает более гибкую и точную работу с системой, позволяя пользователю протестировать разные варианты ответов и продолжить беседу по лучшему сценарию.	

Участники	• **Пользователь** – инициирует отправку сообщения и выбирает модель(и) для обработки • **Система LibreChat** – обеспечивает интерфейс для ввода сообщения, маршрутизацию запроса и отображение полученных ответов	
Триггер	Инициирующим событием является действие пользователя – нажатие кнопки «Отправить» после ввода текста в поле ввода сообщения	
Предусловия	• Пользователь успешно аутентифицирован в системе • Соединение с сервером установлено, а система находится в рабочем состоянии • Текст сообщения прошёл базовую валидацию (не является пустым, соответствует требованиям формата)	
Постусловия	• Сообщение доставлено и отобразилось в пользовательском интерфейсе • Если применялась параллельная обработка, результаты от всех задействованных моделей агрегированы и представлены пользователю • В случае возникновения ошибок, пользователь уведомлён о неудачной отправке с указанием возможных путей исправления.	

Успешный сценарий	• **Ввод сообщения:** Пользователь вводит текст в поле ввода • **Выбор модели(ей):** Пользователь выбирает: • одну конкретную модель, **или** • несколько моделей для параллельной отправки (например, чтобы сравнить результаты) • **Отправка запроса:** Пользователь нажимает «Отправить» • **Валидация:** Система проверяет корректность сообщения и доступность выбранных моделей • **Обработка:** • Если выбрана одна модель, запрос направляется к ней, и система ожидает ответ • Если выбрано несколько моделей, запрос параллельно отправляется ко всем указанным моделям • **Получение ответов:** • В случае одной модели система получает единичный ответ и отображает его • При параллельной отправке система собирает ответы от всех моделей • **Сравнение и выбор ответа (при параллельной отправке):** • Система отображает все полученные ответы • Пользователь сравнивает варианты и выбирает лучший ответ для дальнейшего диалога • **Продолжение диалога:** Пользователь, при желании, может продолжить обсуждение, опираясь на выбранный ответ, либо сохранить несколько ответов для дальнейшего анализа	

Альтернативный сценарий 1 Отправка параллельного запроса нескольким моделям	1. Пользователь вводит текст сообщения в поле ввода LibreChat 2. Пользователь активирует режим «Отправить нескольким моделям» («Параллельный запрос») 3. LibreChat предоставляет список доступных моделей с возможностью выбора нескольких 4. Пользователь отмечает нужные модели и отправляет запрос 5. LibreChat отправляет запрос всем выбранным моделям одновременно 6. LibreChat отображает ответы моделей отдельно, с указанием названия каждой модели 7. Пользователь сравнивает ответы и выбирает наиболее подходящий 8. Пользователь продолжает диалог с выбранным ответом, создавая ветвление диалогов	

UI-Описание	• **Поле ввода сообщения:** Текстовое поле для ввода вопроса • **Список доступных моделей:** • Отдельные иконки или чекбоксы для выбора одной или нескольких моделей • **Кнопка «Отправить»:** Запускает процесс отправки • **Отображение ответов:** • При отправке к одной модели – единственный ответ в списке сообщений • При параллельной отправке – блоки с ответами от каждой модели, позволяющие сравнивать их • **Индикаторы состояния:** Отображают прогресс обработки или ошибки • **Опция ветвления:** • Кнопка или чекбокс «Начать ветвление здесь», позволяющий создать новую ветку диалога на основе выбранного ответа • Возможность вести несколько веток диалога параллельно • **Уведомления и подсказки:** • Системные сообщения об ошибках или успешной отправке • Рекомендации по дальнейшим действиям при недоступности моделей	

AS IS [v.0.1] TO BE []

6.7.2.1.5.2 Выявленная проблематика

6.7.2.2 Сценарии использования LiteLLM [UI]

6.7.2.3 Сценарии использования NoCode aka LangFlow [UI]

6.8 Передача репозитория в деплой (Infra / DevOps)

Подготовка репозитория к деплою и передаче в Infra / DevOps

6.8.1 Цель документа

Данный документ описывает единые требования к оформлению репозиториев сервисов, которые передаются в работу инфраструктурной / DevOps-команде для деплоя, сопровождения и эксплуатации.

Цель:

- сократить время на разбор сервиса инфраструктурой,
- снизить количество уточняющих вопросов,
- обеспечить предсказуемый и повторяемый процесс деплоя.

Документ обязателен к применению для всех новых сервисов и при доработке существующих.

6.8.2 Минимальные требования к сервису

Перед созданием задачи на Infra / DevOps убедитесь, что сервис соответствует следующим требованиям.

6.8.2.1 1. Healthcheck endpoint

Сервис должен иметь healthcheck endpoint.

Требования:

- HTTP endpoint (например: /health)
- возвращает 200 OK, если сервис работоспособен
- не зависит от внешних систем критично (или явно описывает деградацию)

Пример:

GET /health

200 OK

Healthcheck используется:

- для readiness/liveness проверок,
- при деплое,
- для мониторинга.

6.8.2.1.1 ВАЖНО! Необходимо предусмотреть расположение сервиса не в единственном экземпляре

6.8.2.2 2. Метрики (если применимо)

Если сервис отправляет метрики - они должны быть доступны в формате Prometheus.

Требования:

- отдельный endpoint (например: /metrics)
- совместимость с Prometheus scrape

Если метрик нет - это должно быть явно указано в README.

6.8.2.3 3. Алерты (при интеграции с инфровым алертингом)

Если сервис:

- ходит в общий инфровый алертинг,
- или ожидается его мониторинг в production,
- то необходимо описать ожидаемые алерты.

Минимально:

- по каким событиям шлются алерты,
- критичность (warning / critical),
- краткое описание условий срабатывания.

Если алертинг не используется - это также явно фиксируется.

6.8.3 Требования к репозиторию

6.8.3.1 4. Репозиторий с кодом

- код сервиса должен находиться в одном репозитории;
- репозиторий указывается явно в задаче на Infra / DevOps;
- доступы к репозиторию должны быть предоставлены заранее.

6.8.3.2 5. Ссылка на репозиторий в задаче

В задаче на инфраструктуру обязательно указывать:

- ссылку на репозиторий,

- ветку или tag, который предполагается к деплою.

Без ссылки задача не принимается в работу.

6.8.4 Обязательная структура репозитория

В репозитории обязательно должны присутствовать следующие элементы.

6.8.4.1 6. Dockerfile

- Dockerfile для сборки production-образа
- без dev-зависимостей (если не оговорено иное)
- с понятной точкой входа (ENTRYPOINT / CMD)

6.8.4.2 7. Файл зависимостей

В зависимости от стека:

- txt / pyproject.toml
- json

или иной стандартный файл зависимостей

Файл должен:

- быть актуальным,
- позволять воспроизводимую сборку образа.

6.8.4.3 8. README.md (обязательно)

README - ключевой документ для инфраструктуры.

Минимальное содержимое README:

6.8.4.3.1 8.1 Общее описание сервиса

- что это за сервис,
- за что отвечает,
- основные зависимости (БД, кеши, внешние API).

6.8.4.3.2 8.2 Как запустить сервис

- локальный запуск,
- запуск через Docker / docker-compose,
- примеры команд.

6.8.4.3.3 8.3 Production-запуск

- особенности production конфигурации,
- параметры масштабирования (воркеры, ресурсы),
- healthcheck endpoint.

6.8.4.3.4 8.4 Переменные окружения

- список всех ENV-переменных,
- какие обязательны,
- примеры значений,
- дефолты (если есть).

6.8.4.3.5 8.5 Возможные проблемы

- типовые ошибки при старте,
- проблемы с БД / миграциями / внешними сервисами,
- способы диагностики.

6.8.4.4 9. Production docker-compose (если требуется)

Если для деплоя используется docker-compose:

- в репозитории должен быть production-compose файл,
- без dev-override логики,
- с описанными зависимостями и healthcheck.

Если compose не используется - это указывается в README.

6.8.5 Процесс передачи в Infra / DevOps

1. Разработчик / владелец сервиса:
    - оформляет репозиторий по данному документу,
    - проверяет запуск сервиса локально и в Docker.
2. Создается задача на Infra / DevOps:
    - прикладывается ссылка на репозиторий,
    - указывается ветка / tag,
    - кратко описывается цель деплоя.
3. Infra / DevOps:
    - не тратит время на восстановление контекста,
    - использует README как основной источник информации,

- задает вопросы только по нетиповым кейсам.

6.9 Ролевая модель в Langflow (NoCode)

6.9.1 Цели создания ролевой модели

1. Безопасность: защита конфиденциальных данных (API-ключи, промпты, логи) и непрерывной работы продакшен-пайплайнов.
2. Разделение обязанностей: разграничение прав между разработчиками пайплайнов и пользователями.

6.9.2 Открытые вопросы

1. Нужно ли учитывать какую-то ролевку/архитектуру ролевки от Облачной платформы Дюгурова?
2. Нужно ли закладывать ролевку и ChatUI тоже (полагаю, этого ожидают клиенты) или только Langflow?
3. Можем ли мы ограничить доступ к определенным источникам данных про проекту/роли?

Важно! Администрирование ролевой модели будет необходимо отразить во внутренней админке продукта.

6.9.3 Основные сущности

Организация

Отображать название и ID. Уникальность названия не требуется. Создается сервис-менеджером.

Проект

Отображать название и ID. В рамках организации название проекта должно быть уникальным. В рамках всех компаний уникальность не требуется. Проекты заводятся админом организации (см. ниже) (ВОПРОС: стоит ли дать права другим ролям создавать проекты?.

Пользователь

Добавляется в организацию по email админом организации. Админа организации заводит сервис-менеджер либо ранее созданные админы.

Предусмотреть возможность загрузки пользователей через файл.

Группа пользователей

Объединение пользователей, которым можно разом назначить определенную роль. Группа назначается пользователю при создании пользователя или позднее. Требует обсуждения.

Ресурс

Сущности, на которые раздаются права внутри проекта.

1. Новые компоненты
2. API-ключи
3. Глобальные переменные
4. Pipelines (потоки)
5. API-endpoint
6. UI с доступом к pipeline.

Действия над ресурсом

Создание/просмотр/редактирование/удаление ролей/пользователей/проектов/новых компонентов/API ключей/глобальных переменных и т.д..

6.9.4 Роли

Предлагается создать несколько преднастроенных ролей, при этом предлагается роли сделать **редактируемыми**. Админу организации будет доступна таблица, где галочками проставлены права на разные модули/фичи и будет возможность:

- Редактировать галочки
- Добавлять роли.

Список модулей/фичей при этом будет нередактируемый.

Пользователи могут состоять в разных проектах с разными правами.

6.9.4.1 Админ организации

Владелец Nocode внутри организации-клиента. Управление пользователями и настройками.

Права:

- Может завести в организации новых пользователей (высылается приглашение на почту).
- Может назначить другого пользователя админом.
- Имеет права админа на все проекты внутри организации.
- Может назначать права на все проекты внутри организации.
- Может создавать проекты.

6.9.4.2 Админ проекта

Аналог в IT: инженер ML/AI, промпт-инженер, low-code разработчик. Создание, модификация и тестирование потоков (flows) в рамках своего проекта.

Права:

- Создание и редактирование потоков: Полный доступ к конструктору: добавление, настройка, соединение компонентов.
- Управление API-ключами в своем проекте: Добавление, редактирование, удаление ключей (например, OpenAI, Anthropic) в рамках своего проекта. *Не может видеть ключи других проектов.*

- Тестирование потоков: Запуск потоков в режиме "промпт-панели", просмотр результатов и логов своих выполнений.
- Экспорт/Импорт потоков: Сохранение и загрузка конфигураций потоков (`.json`).
- Создание API-эндпоинтов: Публикация потока как API-эндпоинта для внешнего использования.
- Ограниченный просмотр: Видит только те проекты и потоки, в которые его добавили.
- Добавление пользователей в проект: может добавить пользователей с различными правами в проект, но лишь тех, кто заведен в организацию.

6.9.4.3 Тестировщик (Tester / Reviewer)

Аналог в IT: QA-инженер, продукт-менеджер. Тестирование и оценка готовых потоков без возможности их случайного изменения.

Права:

- Просмотр потоков: Возможность открывать и просматривать конфигурацию потоков в режиме "только для чтения".
- Запуск потоков: Активное тестирование потоков через промпт-панель с разными входными данными.
- Просмотр логов выполнений: Анализ результатов и ошибок тестовых запусков.
- Комментирование: Возможность оставлять комментарии прямо на потоке для разработчика.
- Запрещено: Редактирование, удаление, создание новых потоков и управление API-ключами.

6.9.4.4 Пользователь проекта

Аналог в IT: Внешняя система или приложение, конечный пользователь веб-интерфейса. Цель: Только использование развернутых API-эндпоинтов или готовых интерфейсов.
Права:

- Вызов API: Отправка HTTP-запросов на опубликованные эндпоинты.
- Использование готового интерфейса: Взаимодействие с развернутым приложением через предоставленный UI (например, чат-бот).
- Запрещено: Любой доступ к интерфейсу Langflow, просмотр проектов, редактирование, доступ к настройкам.

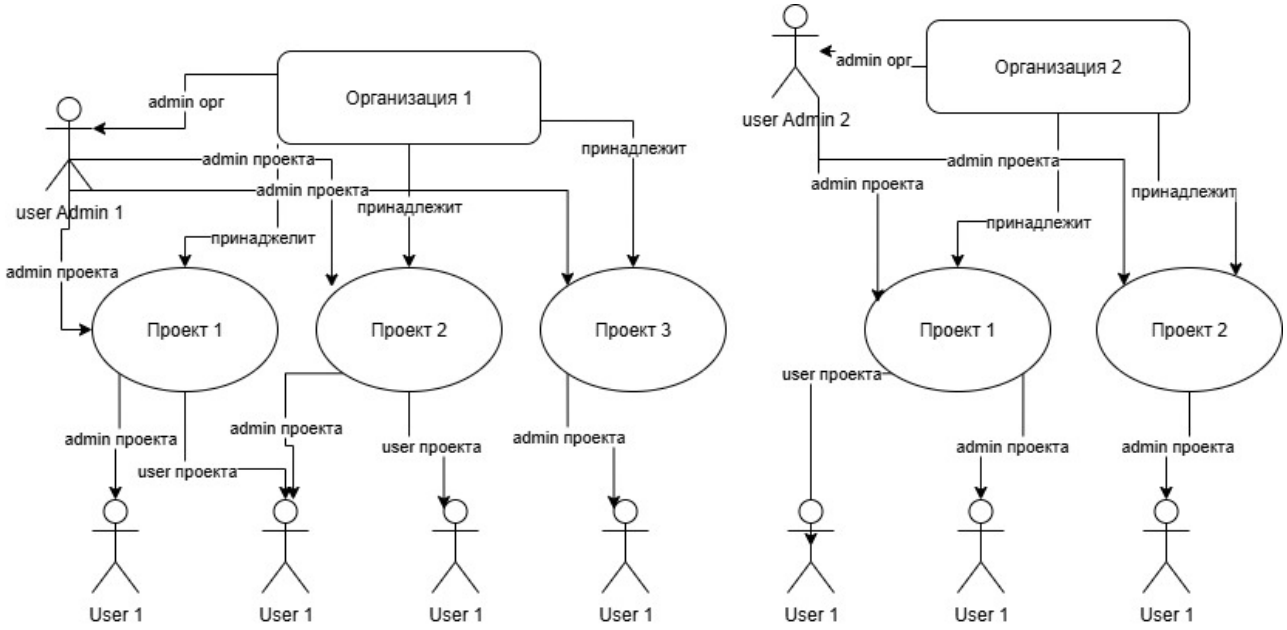
6.9.4.5 Наблюдатель проекта

Сотрудник компании, которого добавили в проект с целью ознакомления. Цель: Только использование развернутых API-эндпоинтов или готовых интерфейсов.
Права:

- Вызов API: Отправка HTTP-запросов на опубликованные эндпоинты.
- Использование готового интерфейса: Взаимодействие с развернутым приложением через предоставленный UI (например, чат-бот).
- Запрещено: Любой доступ к интерфейсу Langflow, просмотр проектов, редактирование, доступ к настройкам.

ТАБЛИЦА ЕЩЕ В РАБОТЕ		Org Admin	Project Admin	Project User	Project Viewer
Управление пользователями	Создание, просмотр, редактирование, удаление пользователей. Назначение им ролей.	+			
Управление проектами (project)	Создание, переименование, архивация, удаление проектов. Назначение пользователей в проекты.	+			
Системные настройки	Настройка глобальных переменных (например, дефолтных API-ключей), параметров сервера, бэкенд-конфигураций.	+			
Просмотр артефактов	Доступ ко всем проектам, потокам, компонентам и API-ключам всех пользователей.	+			
Управление кастомными компонентами	Установка, обновление, отключение кастомных компонентов для всего инстанса.				
Аудит и мониторинг	Просмотр логов выполнения, метрик использования				

6.9.5 Схематичное изображение ролевой модели



6.9.6 RBAC в Langflow с использованием Casbin

6.9.6.1 1. Что сейчас есть

6.9.6.1.1 Как работает авторизация

Сейчас в Langflow все просто - есть флаг **is_superuser** на модели User:

- **is_superuser=True** - полный доступ ко всему
- **is_superuser=False** - видишь только свои ресурсы

Все защищенные эндпоинты (около 40 штук) используют одну FastAPI-зависимость:

```
# src/backend/base/langflow/api/utils.py, строка 35
CurrentActiveUser = Annotated[User, Depends(get_current_active_user)]
```

Функция **get_current_active_user** (файл *src/backend/base/langflow/services/auth/utils.py*, строки 249-252) просто проверяет что пользователь активен и возвращает его. Тип **CurrentActiveUser** используется в **76 местах** в 15 файлах.

6.9.6.1.2 Чего нет

- Организаций и команд

- Проектов как единицы совместной работы
- Гранулярных прав (read/write/execute/delete)
- Групп пользователей
- Аудита действий

6.9.6.1.3 Ключевые модели в коде

User (*src/backend/base/langflow/services/database/models/user/model.py*, строки 25-52):

- id, username, password, is_active, is_superuser
- Связи: api_keys, flows, variables, folders

Folder (*src/backend/base/langflow/services/database/models/folder/model.py*, строки 16-31):

- id, name, description, user_id (FK на User), parent_id (FK на себя)
- Связи: flows, children (вложенные папки)
- Уникальность по паре (user_id, name)

6.9.6.2 2. Выбранный подход

6.9.6.2.1 Почему Dependency Injection Override

FastAPI дает возможность подменять зависимости через **app.dependency_overrides** без изменения исходного кода роутеров. Если подменить **get_current_active_user**, это автоматически затронет все 76 мест где используется **CurrentActiveUser**.

Что это дает:

- Ноль изменений в файлах роутеров
- Минимальный риск конфликтов при мерже с upstream
- Можно включать/выключать RBAC через переменную окружения

6.9.6.2.2 Почему Casbin

- Зрелая библиотека - ее рассматривают сами разработчики Langflow
- Поддерживает RBAC с доменами (права привязаны к проекту)
- Работает с PostgreSQL через async-адаптер
- Проверка прав меньше чем за 1 мс
- Модель авторизации описывается декларативно, политики хранятся в БД

6.9.6.2.3 Почему не другие подходы

Подход	Почему не подходит
Middleware	Та же проблема с маппингом путей. Нет доступа к объекту пользователя на уровне middleware. Конфликты при мерже в main.py
Plugin Architecture	Нужно менять около 30 файлов с эндпоинтами. Высокий риск конфликтов. Забытый хук - дыра в безопасности
Service Layer Proxy	Отдельный сервис-прокси перед Langflow. Добавляет 5-20мс задержки, сложность деплоя и дебага двух сервисов

6.9.6.3 3. Два варианта реализации

Критерий	Вариант А (MVP)	Вариант Б (полный)
Ролевая модель	5 фиксированных ролей, привязка к существующему Folder	Организации, проекты, редактируемые роли, группы
Новых таблиц в БД	2 (user_project_role, casbin_rule)	8 (+organization, rbac_project, role_definition, user_group, user_group_membership, group_project_role)
Новых файлов	около 12	около 30
Файлов Langflow изменено	1 (main.py)	2 (main.py, router.py)
Иерархия	Нет - роли привязаны к Folder напрямую	Организация - Проект - Ресурс
Редактируемые роли	Нет	Да
Группы пользователей	Нет	Да, с каскадным назначением

Критерий	Вариант А (MVP)	Вариант Б (полный)
Импорт пользователей	Нет	Да, CSV/XLSX
API управления RBAC	Минимальный (назначение ролей)	Полный (организации, роли, группы, импорт)
Риск при обновлении upstream	Минимальный	Низкий
Масштабируемость	Ограниченная	Полная

Рекомендация: начать с Варианта А, проверить в проде, потом расширить до Варианта Б. Таблица user_project_role при миграции расширяется, а не заменяется - данные не теряются.

6.9.6.4 4. Вариант А (MVP) - что конкретно делаем

6.9.6.4.1 4.1. Новая таблица user_project_role

Файл: *src/backend/base/langflow/rbac/models/user_project_role.py*

Поля:

- **id** - UUID, primary key
- **user_id** - FK на user
- **folder_id** - FK на folder
- **role_name** - строка (org_admin / project_admin / tester / project_user / observer)
- **assigned_at** - timestamp
- **assigned_by** - FK на user, nullable

Уникальное ограничение на пару (user_id, folder_id).

Alembic-миграция создает эту таблицу плюс **casbin_rule** (стандартная таблица для casbin-sqlalchemy-adapter с полями id, ptype, v0-v5).

6.9.6.4.2 4.2. Конфигурация Casbin

Файл: *src/backend/base/langflow/rbac/casbin_model.conf*

```
[request_definition]
r = sub, dom, obj, act

[policy_definition]
p = sub, dom, obj, act
```

```
[role_definition]
g = _, _, _

[matchers]
m = g(r.sub, p.sub, r.dom) && r.dom == p.dom && r.obj == p.obj && r.act == p.act
```

- **sub** = user_id
- **dom** = folder_id (проект)
- **obj** = тип ресурса (flow, project, api_key и т.д.)
- **act** = действие (read, create, update, delete, execute)

При старте загружаются 5 фиксированных ролей как seed-политики.

6.9.6.4.3 4.3. Модуль Casbin Enforcer

Файл: *src/backend/base/langflow/rbac/enforcer.py*

При старте приложения инициализирует async Casbin Enforcer, подключенный к той же БД что и Langflow. Дает функцию:

```
async def enforce(user_id, folder_id, resource_type, action) -> bool
```

Новые зависимости в **pyproject.toml**:

- casbin>=1.36.0
- casbin-async-sqlalchemy-adapter>=1.4.0

6.9.6.4.4 4.4. EndpointMapper - маппинг URL в ресурс и действие

Файл: *src/backend/base/langflow/rbac/endpoint_mapper.py*

По URL-пути определяет тип ресурса:

Префикс URL	resource_type
/api/v1/flows	flow
/api/v1/projects, /api/v1/folders	project
/api/v1/api_key	api_key
/api/v1/variables	variable
/api/v1/files	file

Префикс URL	resource_type
/api/v1/monitor/messages	message
/api/v1/monitor/builds	build_log
/api/v1/monitor/transactions	transaction
/api/v1/knowledge	knowledge_base

По HTTP-методу определяет действие:

- GET - read
- POST - create (но build/run эндпоинты - execute)
- PATCH/PUT - update
- DELETE - delete

Из URL извлекается UUID ресурса через regex.

Около 15 публичных путей исключены из проверки: /api/v1/login, /api/v1/auto_login, /health, /api/v1/public_flow, /api/v1/starter-projects и др.

При старте приложения сравнивает зарегистрированные роуты FastAPI с известным маппингом. Если найден незнакомый защищенный эндпоинт - выводит предупреждение в лог.

6.9.6.4.5 4.5. ResourceResolver - определение проекта по ресурсу

Файл: *src/backend/base/langflow/rbac/resource_resolver.py*

По типу ресурса и его ID определяет folder_id (проект):

Ресурс	Как определяем folder_id
flow	Запрос в БД: Flow.folder_id
project	folder_id и есть ID проекта
file, message, build_log, transaction	Берем flow_id из пути, потом Flow.folder_id
api_key, variable	Возвращаем None (привязаны к пользователю, не к проекту)

Если вернулся None - пропускаем RBAC, оставляем оригинальное поведение Langflow (owner-only).

6.9.6.4.6 4.6. Dependency Overrides - подмена зависимостей

Файл: *src/backend/base/langflow/rbac/overrides.py*

Три async-функции на замену:

rbac_get_current_active_user (заменяет get_current_active_user):

1. Проверить is_active (401 если нет)
2. Если superuser и LANGFLOW_RBAC_SUPERUSER_BYPASS=true - пропустить RBAC
3. Проверить путь по exempt-списку - пропустить RBAC
4. EndpointMapper определяет resource_type и action
5. ResourceResolver находит folder_id
6. Если folder_id не найден - вернуть пользователя без RBAC
7. Casbin enforce - при отказе 403

rbac_get_current_active_superuser (заменяет get_current_active_superuser):

- Стандартная проверка is_superuser, но дополнительно пускает пользователей с ролью org_admin

rbac_get_current_active_user_mcp - то же что первая, но для MCP-зависимости.

6.9.6.4.7 4.7. Точка интеграции в main.py

Файл: *src/backend/base/langflow/main.py*

После строки 372 (после app.include_router(log_router)), перед exception handler.

Добавляется **7 строк**:

```
import os
if os.getenv("LANGFLOW_RBAC_ENABLED", "false").lower() == "true":
    from langflow.rbac.bootstrap import inject_rbac_system

    @app.on_event("startup")
    async def _rbac_startup():
        await inject_rbac_system(app)
```

Функция inject_rbac_system инициализирует Casbin Enforcer и подставляет три dependency_overrides.

6.9.6.4.8 4.8. API назначения ролей

Файл: *src/backend/base/langflow/rbac/api/members.py*

Метод	Эндпоинт	Что делает
GET	/api/v1/rbac/projects/{folder_id}/members	Список членов проекта с ролями
POST	/api/v1/rbac/projects/{folder_id}/members	Добавить пользователя с ролью
PATCH	/api/v1/rbac/projects/{folder_id}/members/{user_id}	Изменить роль
DELETE	/api/v1/rbac/projects/{folder_id}/members/{user_id}	Удалить из проекта

При добавлении/изменении: запись в user_project_role + Casbin grouping policy (user_id, role_name, folder_id).

6.9.6.4.9 4.9. Структура новых файлов

```
src/backend/base/langflow/rbac/
  __init__.py
  bootstrap.py           - inject_rbac_system()
  enforcer.py            - инициализация Casbin + enforce()
  endpoint_mapper.py     - URL -> (resource_type, action)
  resource_resolver.py   - resource -> folder_id
  overrides.py           - 3 функции подмены зависимостей
  seed.py                - загрузка 5 начальных ролей
  casbin_model.conf      - конфиг модели Casbin
  api/
    __init__.py
    members.py           - эндпоинты назначения ролей
  models/
    __init__.py
    user_project_role.py - SQLModel таблица
```

6.9.6.4.10 4.10. Ограничения Варианта A

- **Нет организаций** - нельзя объединить несколько проектов под единым управлением
- **Нет редактируемых ролей** - 5 фиксированных, для изменения прав нужно менять seed-данные
- **Нет групп** - назначение ролей только по одному пользователю
- **Нет импорта** - добавление только через API
- **api_key и variable** не привязаны к проектам - остается owner-only
- **Нет аудита** - проверки авторизации и изменения ролей не логируются

6.9.6.5 5. Вариант Б (полный) - что добавляется сверх Варианта А

6.9.6.5.1 5.1. Дополнительные таблицы - еще 6 штук

Таблица	Зачем	Ключевые поля
organization	Верхний уровень иерархии	id, name (уникальное), description, created_by
rbac_project	Связь организации с Folder	id, organization_id, folder_id (уникальное), name
role_definition	Редактируемая роль с JSON-правами	id, name, organization_id, is_system, permissions (JSON)
user_group	Группа пользователей	id, name, organization_id
user_group_membership	Связь пользователь-группа	user_id, group_id
group_project_role	Роль группы в проекте	group_id, project_id, role_name

Таблица user_project_role расширяется: project_id теперь ссылается на rbac_project вместо folder.

6.9.6.5.2 5.2. Расширение конфига Casbin

- Правило **g2**: (user_id, "org_admin", organization_id) - привязка админа на уровне организации
- Правило **g3**: (user_id, group_id) - связь пользователя с группой
- В матчерах добавляются OR-условия: org_admin автоматически получает доступ ко всем проектам своей организации
- Нужна кастомная функция get_org_for_project(project_id) через in-memory кэш

6.9.6.5.3 5.3. RoleManager - динамическое управление ролями

Синхронизирует таблицу role_definition с политиками Casbin:

- При назначении роли - читает permissions из role_definition, создает политики в Casbin
- При изменении прав роли админом - удаляет старые политики во всех проектах, создает новые
- Тяжелая операция: при 100 проектах и роли с 20 permissions это около 2000 строк casbin_rule. Лучше делать в транзакции и в фоновой задаче

6.9.6.5.4 5.4. GroupManager - управление группами

Каскадные операции:

- Добавляем пользователя в группу - автоматически получает все роли группы во всех проектах
- Назначаем группе роль в проекте - все члены группы получают эту роль
- Для групп больше 50 человек - batch-операции (add_policies вместо add_policy в цикле)

6.9.6.5.5 5.5. API управления RBAC - 5 роутеров

Роутер	Эндпоинты	Кто может
Организации	CRUD (4 штуки)	superuser / org_admin
Роли	CRUD (4 штуки)	org_admin (системные нельзя удалить)
Члены проекта	CRUD (4 штуки)	project_admin и выше
Группы	CRUD + назначение роли (5 штук)	org_admin
Импорт пользователей	Загрузка CSV/XLSX (1 штука)	org_admin

Регистрируются через одну строку include_router в *src/backend/base/langflow/api/router.py*.

6.9.6.5.6 5.6. Seed Data

При первом запуске миграция создает 5 системных ролей (is_system=True) с предустановленными правами в JSON. При создании новой организации системные роли копируются в нее.

6.9.6.6 6. Матрица ролей и действий

Ресурс:Действие	org_admin	project_admin	tester	project_user	observer
flow:read	✅	✅	✅		✅
flow:create	✅	✅			

Ресурс:Действие	org_admin	project_admin	tester	project_user	observer
flow:update	✅	✅			
flow:delete	✅	✅			
flow:execute	✅	✅	✅	✅	
flow:comment	✅	✅	✅		
project:read	✅	✅	✅	✅	✅
project:create	✅				
project:update	✅	✅			
project:delete	✅				
api_key:read	✅	✅	✅		
api_key:create	✅	✅			
api_key:delete	✅	✅			
variable:read	✅	✅	✅		
variable:create	✅	✅			
variable:update	✅	✅			
variable:delete	✅	✅			
user:read	✅	✅	✅	✅	✅
user:invite	✅	✅			
message:read	✅	✅	✅	✅	✅

Ресурс:Действие	org_admin	project_admin	tester	project_user	observer
message:delete	✅	✅			
endpoint:publish	✅	✅			
knowledge_base:read	✅	✅	✅	✅	✅
knowledge_base:create	✅	✅			
knowledge_base:delete	✅	✅			

6.9.6.7 7. Переменные окружения

Переменная	Умолчание	Что делает
LANGFLOW_RBAC_ENABLED	false	Главный переключатель. При false RBAC вообще не грузится
LANGFLOW_RBAC_DEFAULT_DENY	true	true - блокировать неизвестные эндпоинты (безопаснее). false - пропускать (удобно при разработке)
LANGFLOW_RBAC_SUPERUSER_BYPASS	true	Суперпользователи обходят все проверки RBAC
LANGFLOW_RBAC_CACHE_TTL	300	TTL кэша политик Casbin в секундах (только Вариант Б)

6.9.6.8 8. Какие файлы Langflow меняются

6.9.6.8.1 Вариант А

Файл	Что меняем	Строк
src/backend/base/langflow/main.py	Условный блок RBAC startup после строки 372	+7
pyproject.toml	Добавить casbin и casbin-async-sqlalchemy-adapter	+2

6.9.6.8.2 Вариант Б (дополнительно)

Файл	Что меняем	Строк
src/backend/base/langflow/api/router.py	router_v1.include_router(rbac_router)	+1

Весь остальной код живет в новом пакете **langflow/rbac/** - полностью изолирован от кода Langflow.

6.9.6.9 9. Как проходит запрос (Вариант А)

```
HTTP-запрос
  |
  v
FastAPI резолвит зависимость get_current_active_user
  |
  v  (подменена на rbac_get_current_active_user)
  |
  |- is_active? --Нет--> 401
  |
  |- superuser + bypass? --Да--> return user (без RBAC)
  |
  |- exempt-путь? --Да--> return user (без RBAC)
  |
  v
EndpointMapper.resolve(path, method)
  |
```

```
  |- неизвестный эндпоинт + DEFAULT_DENY=true --> 403
  |
  v
ResourceResolver.resolve(resource_type, resource_id)
  |
  |- folder_id = None --> return user (оригинальное поведение)
  |
  v
Casbin enforce(user_id, folder_id, resource_type, action)
  |
  |- запрет --> 403 Forbidden
  |
  v
return user --> обработчик эндпоинта
```

6.9.6.10 10. Риски и митигация

6.9.6.10.1 Высокое влияние

Риск	Вероятность	Что делаем
Изменение сигнатуры get_current_active_user в upstream	Низкая	CI-тест, который проверяет сигнатуру при каждом мерже из upstream. Функция стабильна - не менялась с начала проекта
Конфликт с будущим нативным RBAC в Langflow	Средняя	Feature flag позволяет мгновенно выключить. Данные в отдельных таблицах - можно мигрировать

6.9.6.10.2 Среднее влияние

Риск	Вероятность	Что делаем
Новые эндпоинты в upstream не попадают в маппинг	Средняя	При старте сравниваем роуты FastAPI с известным маппингом. Логируем предупреждения. DEFAULT_DENY=true блокирует неизвестные

Риск	Вероятность	Что делаем
Рассинхронизация кэша project-org в multi-instance (Вариант Б)	Средняя	Периодическая перезагрузка каждые CACHE_TTL секунд. Для HA - pub/sub через Redis
Хрупкость маппинга путей	Средняя	Маппинг через regex, не точные строки. Валидация при старте

6.9.6.10.3 Низкое влияние

Риск	Вероятность	Что делаем
Overhead Casbin на производительность	Низкая	Меньше 1мс на проверку в памяти. Основная стоимость - ResourceResolver (1 запрос в БД). Кэшируем flow-folder_id
Merge-конфликт в main.py	Низкая	Изменение в конце create_app, далеко от часто меняемых участков
Обновление прав роли - тяжелая операция (Вариант Б)	Низкая	Делать в транзакции, логировать время. В проде - фоновая задача

6.9.6.11 11. Плюсы и минусы выбранного подхода

6.9.6.11.1 Плюсы

- Ноль изменений в коде роутеров - все 76 использований CurrentActiveUser покрыты автоматически
- Минимальные конфликты при мерже с upstream (main.py +7 строк, pyproject.toml +2 строки)
- Feature flag - RBAC включается/выключается без изменений кода
- Type-safe - используем встроенную систему DI FastAPI
- Casbin - та же библиотека что планируют принять разработчики Langflow
- Полная изоляция RBAC-кода в отдельном пакете langflow/rbac/

6.9.6.11.2 Минусы

- Перехват только на уровне зависимостей - нет per-endpoint гранулярности
- Нужно поддерживать маппинг эндпоинтов-ресурсов, который может расходиться с upstream

- ResourceResolver добавляет 1 запрос в БД на каждый запрос (можно кэшировать)
- Фиксированные роли в MVP - для изменения прав нужно менять seed-данные
- api_key и variable в MVP не привязаны к проектам (остается owner-only)
- Нет аудита в MVP

6.9.6.12 12. Миграция с Варианта А на Вариант Б

1. Вариант А задеплоен и проверен в проде
2. Создаем таблицы organization и rbac_project, линкуем существующие Folder
3. Меняем FK folder_id в user_project_role на project_id (FK на rbac_project) - data-миграция сохраняет все назначения
4. Добавляем таблицу role_definition, заполняем из фиксированных ролей
5. Деплоим GroupManager и расширенные API-роутеры
6. Добавляем 1 строку в router.py

Без потери данных. Обратная совместимость. Можно делать по шагам.

6.9.7 Анализ кодовой базы по ролевке LF и оценка предлагаемых решений

6.9.7.1 1. Текущее состояние архитектуры Langflow

6.9.7.1.1 1.1. Технологический стек

- **Фреймворк**: FastAPI (асинхронный веб-фреймворк)
- **ORM**: SQLModel + SQLAlchemy (async)
- **База данных**: PostgreSQL/SQLite
- **Аутентификация**: JWT токены + API ключи
- Архитектура: Монолитное приложение с сервис-ориентированной структурой

6.9.7.1.2 1.2. Текущая система аутентификации и авторизации

6.9.7.1.2.1 Аутентификация

Расположение: src/backend/base/langflow/services/auth/utils.py

Методы:

- JWT токены (access + refresh)
- API ключи
- Auto-login (режим разработки)

Ключевые функции:

- `get_current_active_user()` - главная зависимость FastAPI
- `get_current_active_superuser()` - проверка прав суперпользователя

6.9.7.1.2.2 Авторизация

Текущее состояние: Примитивная двухуровневая система

- `is_superuser = True` - полный доступ
- `is_superuser = False` - доступ только к своим ресурсам

Критическая зависимость:

```
# Файл: /api/utils/core.py, строка 36
CurrentActiveUser = Annotated[User, Depends(get_current_active_user)]
```

Используется во **всех защищенных эндпоинтах**.

В архитектуре отсутствует:

- Совместный доступ к ресурсам
- Команды/организации
- Гранулярные права (read/write/execute/delete)
- Аудит действий

6.9.7.1.3 1.3. Точки интеграции

Компонент	Расположение	Возможность расширения
Middleware Stack	main.py(функция `create_app`)	Высокая - простой стек
Dependency Injection	api/utils/core.py	Высокая - поддержка override
Plugin архитектура	новая директория plugins/	Высокая - добавление собственной системы плагинов
Service Layer	services/	Средняя - требует изменений

Ключевая особенность: FastAPI позволяет переопределять зависимости через `app.dependency_overrides` без изменения исходного кода.

6.9.7.2 2. Требования к решению для форка

Критические требования:

- **Минимальные (желательно) изменения кодовой базы Langflow** - упрощает merge обновлений
- **Изоляция кода RBAC** - логика в отдельных модулях/пакетах
- **Отсутствие конфликтов при мерже** - не затрагивать часто изменяемые файлы
- **Стабильность** - не ломаться при обновлении Langflow

6.9.7.3 3. Четыре подхода к интеграции RBAC

6.9.7.3.1 3.1. Подход 1: Переопределение зависимостей (Dependency Injection Override)

6.9.7.3.1.1 Краткое описание

Отдельный Python пакет, переопределяющий зависимости аутентификации FastAPI в runtime без изменения кода Langflow.

Суть: Использование `app.dependency_overrides` для замены `get_current_active_user` на версию с проверкой RBAC.

Изменения в Langflow: незначительны

Интеграция:

```
from langflow.main import create_app
from langflow_rbac import inject_rbac_system

app = create_app()
inject_rbac_system(app)  # Единственная строка интеграции
```

6.9.7.3.1.2 Преимущества

- Нулевые изменения Langflow - код остается нетронутым
- Полная изоляция - RBAC в отдельном репозитории
- Нет merge конфликтов - обновления проходят без проблем
- Опциональность - легко включить/отключить

- Type-safe - использует встроенную систему FastAPI

6.9.7.3.1.3 Недостатки

- Ограниченные точки перехвата - только на уровне зависимостей
- Runtime overhead - дополнительный слой разрешения зависимостей
- Требуется анализ путей эндпоинтов для определения ресурсов
- Нет автоматического определения действий (read/write) из HTTP методов

6.9.7.3.1.4 Потенциальные риски

- **Низкий риск:** Изменение структуры зависимостей Langflow (вряд ли)
- Средний риск: Необходимость поддержки маппинга эндпоинтов → при обновлении можно пропустить добавление новых эндпоинтов и тогда ролёвка будет работать не везде

6.9.7.3.2 3.2. Подход 2: Middleware-based Authorization

6.9.7.3.2.1 Краткое описание

Middleware перехватывает HTTP запросы, парсит путь/метод для определения ресурса, проверяет права через Casbin.

Суть: Централизованная проверка на уровне HTTP middleware до попадания в эндпоинт.

Изменения в Langflow: незначительны

Интеграция:

```
# В langflow/main.py
from langflow_rbac_middleware import RBACMiddleware
app.add_middleware(RBACMiddleware)  # Добавить после существующих
```

6.9.7.3.2.2 Преимущества

- Централизованная проверка - единая точка авторизации
- Path-based подход - легко аудировать
- Раннее отклонение - блокировка до бизнес-логики
- Отдельный пакет - логика изолирована

6.9.7.3.2.3 Недостатки

- Возможны merge конфликты - main.py часто изменяется
- Сложность парсинга путей - нужен маппинг URL → ресурсы

- Динамические роуты - параметризованные пути требуют поддержки

6.9.7.3.2.4 Потенциальные риски

- Средний риск: Конфликты при merge в main.py
- Средний риск: Изменение структуры URL в Langflow требует обновления парсера
- Низкий риск: Ошибки в парсере могут блокировать валидные запросы
- Высокий риск: Необходимость постоянной синхронизации маппинга путей с API Langflow

---

6.9.7.3.3 3.3. Подход 3: Plugin Architecture

6.9.7.3.3.1 Краткое описание

Форк Langflow с добавлением официальной системы плагинов, RBAC реализуется как плагин.

Суть: Добавление хуков авторизации в ~30 эндпоинтах Langflow + система загрузки плагинов.

Изменения в Langflow: будут затронуты около 30 файлах

Интеграция:

```
# В каждом защищенном эндпоинте
plugin = get_active_plugin()
if plugin:
    await plugin.authorize_request(user, "flow", flow_id, "read")
```

6.9.7.3.3.2 Преимущества

- Максимальная гибкость - хуки в нужных местах
- Чистая абстракция - официальный API плагинов
- Мульти-плагины - можно добавить аудит, логирование

6.9.7.3.3.3 Недостатки

- Много изменений - редактировние эндпоинтов по всей кодовой базе
- Частые merge конфликты - изменения во многих файлах
- Высокая сложность координации - дизайн интерфейса плагинов критичен
- Тестирование совместимости плагинов

6.9.7.3.3.4 Потенциальные риски

- **Высокий риск**: Конфликты при merge в ~30 файлах эндпоинтов (средняя вероятность)
- **Высокий риск**: Рефакторинг эндпоинтов в Langflow потребует обновления хуков

- **Высокий риск**: Сложность поддержки форка при активных изменениях в основном репозитории Langflow
- **Критический риск**: Забытый хук = незащищенный эндпоинт (уязвимость безопасности)

6.9.7.3.4 3.4. Подход 4: Service Layer Wrapper (Proxy)

6.9.7.3.4.1 Краткое описание

Отдельный FastAPI сервис проксирует запросы к Langflow, проверяя права на уровне прокси.

Суть: Архитектура [Client] → [RBAC Service] → [Langflow], общая БД.

6.9.7.3.4.2 Изменения в Langflow: буквально 0 строк

Интеграция: Запуск двух сервисов, клиенты обращаются к RBAC Service на порту 8001.

6.9.7.3.4.3 Преимущества:

- Ноль изменений Langflow
- Независимое развертывание - два процесса
- Легкий rollback - перенаправить на Langflow напрямую
- Общая БД - консистентность данных
- Постепенная миграция - можно роутить частично

6.9.7.3.4.4 Недостатки:

- Запрос проходит через +1 сервис - дополнительная задержка ~5-20ms
- Двойная аутентификация - оба сервиса валидируют токены
- Сложность развертывания - два сервиса для управления
- Сложность отладки - распределенная система
- Ограниченная интерцепция - только HTTP, не внутренние вызовы

6.9.7.3.4.5 Потенциальные риски:

- **Средний риск**: Проблемы синхронизации состояния между сервисами
- **Высокий риск**: Сложность отладки и мониторинга распределенной системы
- **Низкий риск**: Точка отказа - если RBAC service упал, доступа нет
- **Средний риск**: Необходимость поддержки маппинга путей (как в Подходе 2)

6.9.7.4 Выводы

1-й выбор: Подход 1 (Dependency Override)- нулевые изменения, минимальные риски
2-й выбор: Подход 2 (Middleware) - минимальные изменения, приемлемые риски (обсуждаемо)

Подход 3 (Fork+Plugin) - высокие риски конфликтов при мерже, много разработки
Подход 4 (Proxy) - операционная сложность, сложность настройки

6.9.7.5 5. Почему Casbin подходит для RBAC в Langflow

В официальном репозитории Langflow в issues идёт обсуждение о добавлении коллаборативного режима и RBAC. Со стороны разработчиков предлагается использовать Casbin для реализации ролевой модели. Ниже приведена оценка внедрения Casbin для реализации нужной нам ролевой модели, рассмотрены его возможности и сложности, с которыми можно столкнуться при реализации требуемого функционала.

6.9.7.5.1 5.1. Обзор Casbin

Casbin - мощная библиотека управления доступом с открытым исходным кодом, поддерживающая различные модели контроля доступа.

Сайт: https://casbin.org
GitHub Python: https://github.com/casbin/pycasbin

6.9.7.5.2 5.2. Применение ролевой модели в Casbin

Casbin позволяет реализовать предлагаемую ролевую модель организация-проект-пользователь через механизм контекстных политик, где каждое правило авторизации привязывается к конкретному проекту. Пользователь может иметь разные роли в разных проектах (например, быть админом в проекте А и тестировщиком в проекте Б), а админ организации автоматически получает полные права на все проекты внутри своей организации. Группы пользователей реализуются через встроенный механизм групп Casbin, что позволяет назначать роль сразу целой группе без необходимости обновления прав каждого участника по отдельности.

Редактируемые роли требуют дополнительной логики: определения ролей хранятся в базе данных, а при изменении набора прав роли админом происходит автоматическая синхронизация с Casbin - старые правила удаляются, новые добавляются. Специфичные действия (комментирование, публикация эндпоинтов, приглашение пользователей) описываются как отдельные типы действий в политиках Casbin наряду с базовыми операциями чтения, записи и удаления. Благодаря поддержке PostgreSQL и асинхронной работы, Casbin обеспечивает быструю проверку прав (менее 1 миллисекунды) даже при сложной иерархии организаций и проектов.

6.9.7.6 6. Интеграция требований ролевой модели с Casbin

6.9.7.6.1 6.1. Обзор требований

6.9.7.6.1.1 Новая ролевая модель

Сущности:

- **Организация** - верхний уровень иерархии (создается сервис-менеджером)
- **Проект** - уникальность названия в рамках организации
- **Пользователь** - добавляется по email, может состоять в разных проектах
- **Группа пользователей** - объединение для массового назначения ролей
- **Ресурсы** - API-ключи, глобальные переменные, pipelines, endpoints, UI

Роли:

1. **Админ организации** - полные права в организации
2. **Админ проекта** - управление проектом, создание потоков, управление пользователями проекта
3. **Тестировщик** - просмотр и запуск потоков, комментирование (read-only на конфигурацию)
4. **Пользователь проекта** - использование API-endpoints и UI (без доступа к Langflow UI)
5. **Наблюдатель проекта** - аналогично пользователю, но сотрудник компании

Особенности:

- Редактируемые роли (админ может изменять набор прав)
- Иерархия: Организация → Проекты → Ресурсы
- Загрузка пользователей через файл
- Комментирование потоков

6.9.7.6.2 6.2. Сложность реализации на Casbin

Даже с учётом встроенных возможностей Casbin в реализации ролевой модели с его использованием будут сложности.
Почему:

- Требуется **многоуровневая иерархия** (организация → проект → ресурс)
- **Множественное членство** пользователя (разные роли в разных проектах)
- **Динамические роли** с редактируемыми правами
- **Группы пользователей** с каскадным назначением ролей
- **Scope-based авторизация** (права в контексте организации/проекта)

6.9.7.6.3 6.3. Подход к реализации

Исходя из желаемых видов ролей и их связей оптимальным вариантом реализации видится **гибридный подход RBAC + Scoped Policies** (политики с привязкой к проекту): роли назначаются в

контексте конкретного проекта, а политики определяют права роли на типы ресурсов внутри этого проекта. При проверке прав Casbin учитывает роль пользователя в проекте, права этой роли на требуемое действие, и принадлежность ресурса проекту. Админ организации получает специальную обработку с автоматическим доступом ко всем проектам организации. Группы пользователей реализуются через встроенный механизм групп Casbin g3 (способ связать пользователя с группой, а группу с ролью), что позволяет назначить роль сразу группе и автоматически применить её ко всем участникам.

6.9.7.6.4 6.4. Основные компоненты реализации

Расширенная модель данных: Потребуется создание сущностей Organization, Project, UserProjectRole, UserGroup, RoleDefinition для хранения иерархии и настроек ролей.

Адаптация Casbin: Создание обертки ScopedEnforcer (модуль, проверяющий права в рамках конкретного проекта) с методами проверки прав в контексте проекта, получения ролей пользователя и определения принадлежности ресурсов.

Редактируемые роли: Хранение определений ролей в БД с синхронизацией политик Casbin при изменении прав админом через специальный RoleManager.

Группы пользователей: Компонент UserGroupManager (инструмент для управления группами) для создания групп, добавления участников и назначения ролей с автоматическим применением прав ко всем участникам группы.

Специфичные действия: Расширение набора действий помимо CRUD (comment, execute, publish, invite, manage_keys) с проверками в соответствующих эндпоинтов.

6.9.7.7 7. Заключение

Casbin как решение:

- Разработчики Langflow планируют использовать Casbin
- Идеально подходит для RBAC
- Поддержка PostgreSQL и async
- Высокая производительность
- Зрелое и проверенное решение

6.9.7.7.1 7.3. Выводы по расширенной ролевой модели

Ключевые выводы:

1. Casbin **способен** реализовать требуемую модель, но требуется значительная кастомизация
2. Рекомендуется **гибридный подход** (RBAC + Scoped Policies) для баланса гибкости и сложности
3. **Редактируемые роли** требуют дополнительной синхронизации между БД и Casbin

Что потребуется:

- Расширенная модель БД (Organization, Project, UserProjectRole, UserGroup, RoleDefinition)
- Кастомный `ScopedEnforcer` с поддержкой проверки иерархии
- `RoleManager` для динамического управления правами ролей

- `UserGroupManager` для каскадного назначения прав
- API для управления организациями, проектами, пользователями, ролями
- Дополнительные действия (comment, publish, invite, manage_keys)

6.9.8 Проектирование UI-ролевки для NoCode

6.9.8.1 Ссылка на макет

Sorry, the widget is not supported in this export.
But you can reach it using the following URL:

https://www.figma.com/design/rCUeKMHGfdbMGC2iRsoEef/Untitled?node-id=0-1&p=f&t=Cvlzov2uWUjry4Ql-0

https://www.figma.com/design/rCUeKMHGfdbMGC2iRsoEef/Untitled?node-id=0-1&p=f&t=Cvlzov2uWUjry4Ql-0

6.9.8.2 Введение

Платформа «NoCode» - визуальный low-code конструктор на основе фреймворка LangChain, который позволяет быстро создавать, тестировать и развертывать приложения, работающие с большими языковыми моделями (LLM), используя интерфейс Drag-and-Drop, и значительно упрощает разработку ИИ-агентов и сложных рабочих процессов без написания большого количества кода.

Однако платформа ограничена: у каждого пользователя — только собственные проекты и потоки. Более того, на данный момент пользователь не может приглашать других в свои проекты — совместная работа возможна лишь через передачу собственных учетных данных (логина и пароля).

Этот подход создаёт риски:

- Невозможно отследить, кто именно вносил изменения.
- Нет возможности ограничить действия гостя: любой, получивший доступ, получает полный контроль над всеми проектами, API-ключами и настройками.
- При утрате доверия к участнику приходится менять пароль — это нарушает непрерывность работы.
- Нарушаются базовые принципы информационной безопасности: конфиденциальность, мониторинг и аудит, минимизация привилегий.

Таким образом, введение ролевой модели кардинально повысит безопасность, удобство совместной работы и соответствие современным ожиданиям пользователей.

Без этого платформа остаётся инструментом только для индивидуального использования, а любая форма коллаборации вынужденно происходит в обход принципов безопасного доступа — через совместное использование учётных данных.

6.9.8.3 Основные принципы проектирования

6.9.8.3.1 Минимализм

UI ролевой модели проектируется по принципу **минимальной достаточности**: он предоставляет только те элементы управления, которые необходимы для решения ключевых задач — безопасного делегирования доступа и разделения обязанностей — без избыточной сложности:

- **Преднастроенные роли**. Пользователь выбирает из понятных, семантически названных ролей («Админ проекта», «Тестировщик», «Наблюдатель»), а не настраивает десятки разрешений вручную.
- **Ограниченный набор редактируемых прав**. Админ организации может корректировать роли, но только в рамках фиксированного списка модулей («Управление пользователями», «Потоки», «API-ключи» и т.д.), чтобы избежать хаотичной кастомизации.
- **Контекстно-зависимый интерфейс**. Langflow и другие компоненты автоматически скрывают недоступные действия — пользователь не видит кнопок, которые всё равно не может нажать.

6.9.8.3.2 Иерархия управления доступом

Платформа NoCode будет строится на трёхуровневой иерархической модели управления доступом, обеспечивающей гибкость и безопасность:

6.9.8.3.2.1 *Уровень – Организация*

Это корневая сущность, представляющая юридическое лицо, команду или клиентскую учётную запись.

Каждая организация имеет:

- Уникальный идентификатор;
- Название (не обязательно уникальное в глобальном масштабе);
- Список пользователей, принадлежащих этой организации;
- Список проектов, созданных в рамках организации.

Организацию может создать только **сервис-менеджер** (технический администратор платформы).

Внутри организации назначается как **минимум один админ организации**, который управляет пользователями и проектами.

Пользователь может принадлежать к нескольким организациям одновременно.

6.9.8.3.2.2 *Уровень – Проект*

Это логическая единица работы, объединяющая связанные между собой LLM-потоки, API-ключи, эндпоинты и интерфейсы.

Проект всегда принадлежит ровно **одной организации**.

В рамках одной организации **названия проектов** должны быть **уникальными** (глобальная уникальность не требуется).